- В дизайнере Alpha.HMI (свойства X, Y, Ширина (Width) и Высота (Height)).
- В режиме рантайма, обращаясь к свойствам X, Y, Ширина (Width) и Высота (Height).

- Удалены функции DisplayCurrentEventsSnapshot и DisplayCurrentEvents, используемые для включения и отключения режима снимка. Вместо них теперь введена новая функция SetSnapshotState.
- Удалена функция GetEventParam.

Добавлены новые типы:

- **EventStatistics**. Не отображается на форме и предназначен для вывода детальной статистики по событиям, которая раньше выводилась в статусной строке. В тип **Alarms_Table** добавлена ссылка на экземпляр типа **EventStatistics**, куда записываются данные статистики.
- **Alarms_Sounds**. Не отображается на форме и предназначен только для воспроизведения звуков событий.
- **Alarms_Table**. Представляет собой таблицу сообщений о событиях без панели инструментов, статусной строки и возможности проигрывания звуков событий.
- Разрешения пользователя (**Permissions**). Не отображается на форме и предназначен для ограничения прав пользователей.

Другие изменения:

- Переработаны настройки в окне **Параметры**.
  - Вкладка **Важность событий** переделана на **Звуки событий**.
  - На вкладке **Вид таблицы** добавлены возможности:
    - скрывать заголовки столбцов;
    - включать сортировку по заголовкам;
    - включать перенос строк;
    - включать отображение миллисекунд;
    - включать слежение за последними событиями;
    - разрешать выбор нескольких строк в таблице;
    - скрывать полосы прокрутки.
  - Вкладка **Механизмы очистки** переделана на **Подключение**.
  - Добавлена вкладка **Фильтр отображения** для создания словаря для полей фильтров, а также для выбора столбцов, которые будут отображаться в окне **Фильтр пользователя**.
  - Добавлена вкладка **Квитирование**.
- Файл сессии приложения `session.json` переименован на `app_session.json`.
- Ликвидирован тип **UnacknowledgedEventsButton**, приводящий к зависаниям.
- Добавлена возможность ограничения прав пользователей на действия в столбце **Источник события**. Для этого в типе **Разрешения пользователя** (**Permissions**) есть свойство Разрешить действия в столбце источник события (ActionInSourceColumn).
- Все свойства экземпляра типа **Alarms** (кроме ActiveMode) вынесены на кладки окна параметров.
- Переработаны правила настроек экземпляра типа **Alarms**.
- Теперь экземпляр можно настроить:
  - Через функцию LoadConfiguration.
  - Задать свойства экземпляра в дизайнере. При первой загрузке настроек из файлов значения используются как начальные и далее сохраняются по команде пользователя.
  - Обращаться к свойствам экземпляра в рантайме.

Исправленные ошибки

- В историческом режиме отображались оперативные сообщения.
- Устранён ошибочный пробел в имени свойства ClearSoundQueryTag.
- Не блокировалась кнопка запроса при некорректном интервале запроса.
- Исправлены причины зависания Alpha.HMI при ведении детальной статистики.
- Функция SetAdvancedRequestFilterAsJson не возвращала ошибку при неверном выражении фильтра.
- Фильтр, установленный функцией RequestRecentEvents, применялся и при запросе истории.
- При фильтрации по столбцу **Группа важности** таблица предпросмотра экспорта оказывалась ошибочно пуста.
- Исправлены прочие некритичные ошибки.

3.0.1

Исправленная ошибка

Устранена причина, из-за которой не было возможности менять ширину столбцов при включенном разрешении на изменение (свойство Возможность изменения вида пользовательского интерфейса установлено в **«true»** и у пользователя есть право **ChangingView**).

2.5

Новые возможности

- Типу **Alarms** добавлена ссылка на команду OnHistoryLoad, которая вызывается по окончании загрузки истории и после подгрузки истории в оперативный режим.
- Теперь последняя использованная папка запоминается и загружается при следующем запуске Alpha.HMI.Alarms. Такое возможно только если есть доступ к файловой системе.
- Добавлены новые права пользователей, используемые при работе с подсистемой безопасности:
  - Выбор принтера (PrinterSelecting) для включения разрешения сменить принтер. Право добавлено вместо существующего права FileSystemAccess.
  - Принтеры (Printers), содержащее списки разрешенных и запрещённых принтеров.
- Теперь последний используемый принтер сохраняется и предлагается при перезапуске окна печати.
- Добавлена возможность настройки состава фильтра отображения.
- Для типа **Alarms** добавлены новые свойства:
  - Папка для экспорта и импорта (ExportImportFolderPath), позволяющее задавать папку экспорта событий.
  - Режим фильтрации источника событий (SourceFilterMode), позволяющее выбрать какие события (все или только по передаваемому источнику) будут отображаться в оперативном и историческом режиме.

Улучшения

- Теперь SetAdvancedFilter и фильтр отображения "умеют" работать с операторами OR и AND.
- Ранее при фильтрации по признаку квитированности отображались только квитированные события. Теперь же отображаются еще и события, не требующие квитирования.
- Обновлён механизм экспорта для совместимости с Alpha.HMI.Tables 2.0.9.
- Добавлена возможность произвольной настройки колонтитулов экспорта и печати с помощью редактора колонтитулов.

Исправленные ошибки

- В веб-версии исправлено смещение списка столбцов в настройках вида таблицы.
- Ранее при отключенных правах на фильтрацию блокировался также доступ к хронологии и предустановленным интервалам. Теперь такая ошибка исправлена.
- Исправлена ошибка, из-за которой в строке состояния отсутствовало предупреждение о том, что тег сигнала очистки очереди звуков задан неверно.
- Исправлена ошибка, из-за которой функции AckAll и AckSelected экземпляра типа **Alarms** не работали сразу после вызова функции LogOn.
- Устранена ошибка, возникающая при выводе детальной статистики по важности вместе с установленной фильтрацией по квитированным событиям.
- Устранена причина, по которой не удавалось получить идентификатор объекта при действии в столбце **Источник события**.
- Исправлены прочие некритичные ошибки.

2.5.1

Исправленная ошибка

Исправлено зависание Alpha.HMI.Viewer при большом количестве событий и включенном свойстве Отображать детальную статистику по важности.

2.5.2

Исправленные ошибки

Исправлены ошибки в работе функции SetAdvancedRequestFilterAsJson, из-за которых активные события запрашивались, но не отображались в оперативном режиме.

2.5.3

Исправленная ошибка

Исправлена ошибка, из-за которой при включенном режиме снимка и свойстве LookLatestEvent со значением **«true»** вновь сгенерированные события возвращали журнал в начало списка событий.

Изменения документации

Редакция 1

Справочное руководство

Обновлено описание функций SetAdvancedFilterAsJson (стр. 1) и SetAdvancedFilter (стр. 1).

Редакция 2

Руководство пользователя

Обновлено описание печати списка сообщений о событиях (стр. 46).

2.4

Новые возможности

- Добавлена возможность задать цвета панели инструментов при помощи темы оформления.
- Добавлена возможность задать цвета контекстного меню при помощи темы оформления.

Исправленные ошибки

- Устранена причина, по которой данные событий OnSoundPlayed и OnExport помещались в одну переменную - EventParams [0], из-за чего данные одного события могли подмениться данными другого события. Теперь данные события OnSoundPlayed перемещены в переменную EventParams [10], и путь до звукового файла относительно папки ресурсов проекта по событию OnSoundPlayed можно теперь получить с помощью функции GetEventParam(10).
- Исправлены ошибки работы функции SetAdvancedFilter со словами, заканчивающимися на "НЕ".
- Решена проблема, из-за которой при наличии переноса строк вместо текущей страницы печаталось две страницы.
- Устранены зацикливания алгоритмов, из-за которых тормозило перемещение скролла мышью.
- Исправлены прочие ошибки

2.4.1

Новая возможность

Теперь можно отключить требование воспроизведения звука `default.wav`, который должен звучать по умолчанию, если никакой звук для события не указан. Для этого каждой группе важности добавлен флаг **Принудительное воспроизведение**.

Исправленные ошибки

- Исправлена ошибка, из-за которой пользовательский столбец не заполнялся данными в историческом режиме.
- Исправлена ошибка, из-за которой не удавалось выполнить запрос истории и применить фильтр отображения в одном скрипте.
- Исправлены прочие ошибки.

Изменения документации

Редакция 1

Внутренние изменения. Содержимое документа не изменилось.

2.3

Новые возможности

- Информацию о применённом фильтре запроса теперь можно добавить в файл экспорта и печати. Для этого в окне предпросмотра экспорта и в окне предпросмотра печати добавлен флаг **Фильтр запроса**.

- Добавлена возможность получения и вывода в журнал текста ошибки при неуспешном экспорте.

  Для этого нужно экземпляру типа **Alarms** добавить команду Action_OnExport с кодом обработчика в событии Invoked:

```
DebugTool_1.Log("OnExport: path: " +GetEventParam(0));
DebugTool_1.Log("OnExport: Error: " +GetEventParam(4));
```

  А в свойстве OnExport экземпляра типа **Alarms** сослаться на добавленную команду Action_OnExport.

Улучшения

- Теперь при запуске приложения по ярлыку всегда выполняется принудительная компиляция проекта и автоматическое обновление выходной папки с результатом компиляции. Таким образом, теперь не нужно вручную очищать выходную папку для применения изменений, сделанных в проекте.
- Ранее состояние флага **Текст** из окна экспорта не сохранялось ни в какой в конфигурационный файл, а введенный текст сохранялся в файл с настройками печати (`print.json`). Теперь же состояние флага **Текст** и сам текст сохраняются отдельно от настроек печати - в конфигурационный файл `export.json`.

Исправленные ошибки

Исправлены некритичные ошибки.

Изменения документации

Редакция 1

Руководство пользователя

- Актуализированы некоторые скриншоты.
- Описано как запускать приложение в ОС Linux (стр. 12).

2.2

Новые возможности

- Реализован ввод даты и времени в фильтре отображения.
- Добавлена возможность задавать допустимое количество листов для печати.
- Реализована возможность блокировки и подавления событий.

Улучшения

Теперь при включении принудительного слежения за последним событием или слежения за последним поступившим событием слежение выполняется и за верхней и за нижней строкой.

Исправленные ошибки

- Решена проблема с потерей выделения строк в настройках вида таблицы.
- Исправлена ошибка, из-за которой было невозможно задать фильтр отображения по столбцу **Тип события**.
- Исправлены другие некритичные ошибки.

2.1

Новые возможности

- Стало удобнее добавлять стандартные столбцы. Теперь при добавлении нового столбца можно выбрать идентификатор из готового списка. Тип и описание стандартного столбца подгружаются автоматически.
- Реализована возможность загрузки настроек приложения из файла `app_settings.json`. Пока из файла можно загрузить только параметры работы с экранной клавиатурой.

  Файл хранится по пути:

  - ОС Windows: `C:\ProgramData\Automiq\HMI.Alarms`
  - ОС Linux: `/home/<user>/Automiq/HMI.Alarms`
- Добавлена возможность вызова экранной клавиатуры.

  Вызвать клавиатуру возможно:

  - При встраивании типа **`Alarms`** в проект.
  - В Alpha.HMI.Alarms как отдельном приложении при использовании настроечного файла `app_settings.json`.
- Появилась возможность настройки форматов времени/даты для столбцов таблицы.

Улучшения

- На панели запроса исторических данных добавлены кнопки **Предыдущий интервал** и **Следующий интервал**.
- Изменено расположение элементов в окне настройки стиля события. Теперь слева находятся поля, управляющие условием применения декорирования, а справа - поля значений.
- Теперь текстовая подсказка к ошибке всегда принудительно отображается рядом с полем ввода, содержащим ошибку.

Исправленные ошибки

- Исправлена ошибка, из-за которой при печати страница разбивалась на два листа. Теперь при печати нет автоматического подбора высоты строк для переноса слов, и высота строк не увеличивается.
- Устранены причины, по которым при использовании разрешения 1024*768 диалоговые окна выходили за пределы экрана.
- Использование некорректных типов в столбцах приводило к возникновению исключения. Теперь типы столбцов float4/float8 заменены на float/double.
- Устранены причины, приводившие к тому, что темы оформления не применялись к некоторым диалоговым окнам.
- Исправлены другие некритичные ошибки.

2.1.1

Новые возможности

В окне предпросмотра экспорта добавлены возможности:

- Вводить произвольный текст, который будет отображаться в начале файла экспорта (все форматы: `*.csv`, `*.xlsx`/`*.xls`, `*.pdf`). Теперь можно указывать, например, имя объекта, к которому относятся данные в файле.
- Включать отображение информации о примененном фильтре в начале файла экспорта (все форматы: `*.csv`, `*.xlsx`/`*.xls`, `*.pdf`).

Исправленные ошибки

- Исправлена ошибка, из-за которой при отключенном праве просмотра истории кнопка **Исторический режим** все равно была доступна.

- Исправлены другие некритичные ошибки.

2.1.2

Изменение

Теперь на листе печати вместе с количеством отображаемых сообщений выводится также общее количество сообщений.

Исправленная ошибка

При многократном нажатии на кнопку печати аварийно завершалась работа Alpha.HMI. Во избежание повторения ситуации все кнопки и элементы окна предпросмотра теперь блокируются на время печати.

2.1.3

Изменение

Условия фильтрации, заданные с помощью функции SetAdvancedFilter, теперь автоматически подставляются в окно **Фильтр отображения**. Ранее при наличии расширенного фильтра (заданного с помощью функции SetAdvancedFilter) поля условий пользовательского фильтра (окно **Фильтр отображения**) оставались пустыми.

Изменения документации

Редакция 1

Руководство пользователя

Актуализированы некоторые скриншоты.

Справочное руководство

- Заменен пример использования (теперь пример компилируется) для функции SetHistoricalInterval (стр. 1).
- Обновлено описание свойства AcknowledgmentAvailable (стр. 1).

2.0

Новые возможности

- Добавлена возможность отображать описание корневого объекта из пользовательского атрибута в пользовательском столбце.
- Реализован запрос исторических данных по времени срабатывания. Для этого используйте выпадающий список кнопки **Запросить данные**.
- Добавлена возможность установки критериев запроса исторических данных (новое свойство Расширяющий интервал (HistoryExpandInterval)) для экземпляра типа **Alarms**.
- Добавлена панель с отображением прогресса печати и блокировка кнопок на время печати.

- Добавлена возможность отключения сведения событий. При отключенном сведении события деактивации помещаются в отдельные строки и не объединяются с событиями активации. Для отключения сведения экземпляру типа **Alarms** добавлено свойство **Режим сведения событий** (**MergeEvents**) со значениями:
  - **«0»** - полное сведение;
  - **«1»** - частичное сведение;
  - **«2»** - отключенное сведение.

  Для отображения типа события (активация, квитанция квитирования, псевдоактивация, деактивация) и времени предшествующей активации в Alpha.HMI.Alarms теперь можно добавлять пользовательские столбцы с идентификаторами kind и prev_activation_time.
- Реализовано мгновенное отключение звука после квитирования события.
- Добавлена возможность отображения детальной статистики в строке состояния исторического режима (новое свойство Отображать детальную статистику по важности (DisplaySeverityStatistics) для экземпляра типа **Alarms**).
- Добавлена возможность ограничения срока отображения оперативных событий. Для этого используйте узел **Механизмы очистки** в окне **Параметры**.
- Добавлена возможность подгрузки исторических алармов в оперативном режиме с ограничением по количеству сообщений.
- Добавлена возможность ограничения максимального количества событий (новое свойство Максимальное количество событий (MaxVisibleEvents) со значением по умолчанию - **«4294967295»**).
- Добавлена возможность просматривать графики сигналов-источников событий. Для просмотра графика используйте команду **Открыть график** в контекстном меню. Команда становится доступной при выделении одной и более строк в таблице.
- В окне печати теперь можно указывать произвольный текст, который будет отображаться в верхнем колонтитуле на листе печати. Поле ввода можно использовать, например, для указания наименования объекта.
- Добавлена возможность скрывать команды выпадающего меню кнопки **Запросить данные**. Для этого используйте флаги **Предустановленные интервалы**, **Фильтр запроса**, **Выбор хронологии** на узле **Панель инструментов** окна **Параметры**:
  - Флаг **Предустановленные интервалы** отвечает за видимость команд **Последние 10 минут**, **Последние 30 минут**, **Последний час**, **Последние 6 часов**, **Последние 12 часов**, **Последние сутки**.
  - Флаг **Фильтр запроса** отвечает за видимость команд **Без фильтра** и **Запрос с фильтрацией**.
  - Флаг **Выбор хронологии** отвечает за видимость команд **По времени генерации** и **По времени срабатывания**.

Изменения

- Теперь при переходе между режимами исторический интервал не сбрасывается, а сохраняется.
- При выключенном сведении в строке, соответствующей событию квитирования, значение не выводится.

Исправленные ошибки

- Решены проблемы с применением темы оформления.
- Исправлена ошибка, из-за которой неверно отображались данные в окне **Сортировка**.
- Исправлена ошибка, из-за которой некорректно применялась фильтрация по дате и времени.
- Устранена причина, по которой не отображалось контекстное меню для пустой таблицы.
- Теперь функция SetAdvancedFilter корректно работает с оператором NOT.
- Исправлены ошибки в работе фильтра отображения и функции SetAdvancedFilter, из-за которых отрицательные значения становились положительными.

- Теперь при работе с большим числом сообщений (более 40 тыс.) квитирование выполняется значительно быстрее.
- Исправлены другие некритичные ошибки.

2.0.1

Изменение

Принудительное слежение за последним событием и слежение за последним поступившим событием теперь означают слежение за верхней строкой при любой сортировке.

2.0.2

Исправленные ошибки

- Устранена причина, по которой для свойства Имя приложения с правами безопасности не применялось значение, отличное от **«Alarms»**.
- Исправлена ошибка, из-за которой при печати одна страница разбивалась на две.

Изменения документации

Редакция 1

- Обновлены скриншоты в разделе с описанием встраивания Alpha.HMI.Alarms в проект (стр. 8).
- Обновлены некоторые скриншоты по всему документу.
- Исправлена навигация в разделе с описанием звукового сопровождения событий (стр. 24).

Редакция 2

Внутренние изменения. Содержимое документа не изменилось.

1.3

Новые возможности

- Автоматическое масштабирование листа для печати с учетом размеров столбцов.
- Ручное добавление пользовательских столбцов таблицы сообщений о событиях. С пользовательскими столбцами можно выполнять все те же действия, что и со стандартными столбцами.

Изменения

- Для приложения Alpha.HMI.Alarms добавлена иконка.
- Теперь фильтрация в таблице сообщений регистронезависима.
- Свойство HideAckedEvents (Не отображать неактивные квитированные события) объявлено устаревшим и будет удалено в следующих версиях. Вместо указанного свойства используйте настройку **AckRequired** в Alpha.Server.
- Свойство ShowRelativeTag (Отображать относительный тег источника сообщения) объявлено устаревшим и будет удалено в следующих версиях. Теперь для указания тега родительского узла источника события добавляйте пользовательский столбец с идентификатором **«area_path»**.

Исправленные ошибки

- Исправлена ошибка, из-за которой выделялись не те сообщения, по которым кликали мышкой.
- Решены проблемы, приводящие к квитированию не тех сообщений, которые были выделены.
- Исправлена ошибка, из-за которой не применялось значение свойства **Не отображать неактивные квитированные события**, заданное скриптом.
- Устранена причина, по которой для пользователя без доступа к оперативному режиму (запрещено право **`ViewCurrentEvents`**) при запуске ошибочно устанавливался оперативный режим.
- Решены проблемы с отображением комментария и времени генерации для событий, не требующих квитирования.
- Исправлена ошибка при работе с двумя клиентскими экземплярами, из-за которой при выходе из одного клиента пользователь (не имеющий прав на квитирование) мог квитировать событие во втором клиенте.
- Исправлено зависание при выделении слишком большого числа сообщений в таблице.

1.3.1

Внутренние изменения. Функциональность компонента не менялась.

1.3.2

Исправленные ошибки

- Исправлена ошибка, из-за которой при печати большого числа страниц зависал проект Alpha.HMI.
- Свойство **Принудительное слежение за последним событием** (**FollowLatestEvent**) больше не отключает возможность выделения строк в историческом режиме.
- Решена проблема, из-за которой при неверно указанном пути экспорта не выводилось сообщение об ошибке. Теперь при неверно указанном пути экспорта и невозможности автоматически создать указанную папку появляется сообщение об ошибке. Если же указан несуществующий путь с существующим разделом диска, то папка будет автоматически создана при экспорте.

Изменения документации

Редакция 1

Внутренние изменения. Содержимое документа не изменилось.

Редакция 2

Руководство пользователя

Переработан раздел **«Экспорт сообщений в файл»**:

- Описано сохранение сообщений в архив формата `*.avz` и загрузка такого архива.
- Описана возможность предпросмотра таблицы и настройки ширины столбцов при экспорте.
- Указаны пути сохранения экспортируемых файлов для ОС Winows и ОС Linux.

Редакция 3

Руководство пользователя

- Описан запуск в виде отдельного приложения для ОС Linux (стр. 12).

Справочное руководство

- Пример использования SetHistoricalInterval (стр. 1) заменен на рабочий.
- Переработано описание работы с функцией SetAdvancedFilter (стр. 1). Добавлено много новых примеров использования функции.
- Добавлена пометка о том, что свойство Не отображать неактивные квитированные события (стр. 1) (HideAckedEvents) устарело.

1.2

Важно. Текущая версия предназначена для использования в Alpha.HMI 1.9.0.
Новые возможности

- Теперь в окне выбора цвета группы событий можно задавать десятичный или шестнадцатеричный код цвета в модели RGB.
- Условия фильтра отображения теперь проверяются во время ввода.
- Добавлена возможность комбинировать стили шрифта на вкладке **Вид таблицы** окна настроек.
- Максимальное количество сообщений в таблице увеличено до **«2147483647»**.
- Добавлена возможность настройки ориентации страницы перед печатью и экспортом в `.pdf`.
- Появилась возможность установки темной или светлой темы оформления (новое свойство Тема оформления для экземпляра типа **Alarms**).

Изменения

- Обновлены окна авторизации пользователя, смены пароля, настройки печати и окно предварительного просмотра.
- В окне авторизации и смены пароля кнопка изменения раскладки клавиатуры заменена на индикатор текущей раскладки.
- Оптимизировано число столбцов на вкладке **Настройка событий** окна настроек:
  - Столбцы **Фон**, **Текст** и **Пример** заменены на столбец **Сообщение**.
  - Удалены столбцы **Фон мигания** и **Текст мигания**.
  - Столбцы **Фон квитирования** и **Текст квитирования** заменены на столбец **Квитирование**.

Исправления

- Исправлено неверное выделение строк в настройках вида таблицы.
- Устранён сбой в работе метода LoadConfiguration.
- Верные условия фильтра запроса считались некорректными при отсутствии соединения с сервером.
- Некорректно выполнялась сортировка в столбце **Квитировано**.
- Исправлены некритичные ошибки, возникавшие при работе в ОС Linux.

1.2.1

Новые возможности

Появилась возможность одновременного выбора нескольких предустановленных фильтров (выпадающий список **Одновременное использование** в узле **Фильтры** окна **Параметры**).

Изменения

- Теперь для события по подусловию с атрибутами Sound=' ' и SoundEnabled='1' звук `default.wav` не проигрывается.
- Звуковые файлы теперь хранятся отдельно от папки с конфигурационными файлами. Полный путь к звукам относительно папки проекта: `...\resources\Sounds\*.wav`.
- Конфигурационные файлы теперь хранятся по пути `C:\ProgramData\Automiq\HMI.Alarms`.
- При экспорте файлы теперь сохраняются по пути `C:\ProgramData\Automiq\HMI.Alarms\Export`.

Исправления

- Исправлены ошибки порядка воспроизведения звуков по приоритетам.
- Исправлена ошибка, приводящая к зависанию дизайнера Alpha.HMI сразу после перехода в режим исполнения.
- Исправлены другие некритичные ошибки.

1.2.2

Новые возможности

- Реализована возможность отправки команд квитирования на ПЛК. Для экземпляра типа **Alarms** добавлено новое свойство OnAck. Из свойства OnAck можно сослаться на заранее созданную пользователем команду, скрипт которой будет выполняться при квитировании.
- Появилась возможность запуска Alpha.HMI.Alarms в виде отдельного приложения по ярлыку из Пуск. Теперь Alpha.HMI.Alarms можно встраивать в проект автоматизации или использовать как отдельное приложение.
- Добавлена возможность разрешения/запрета выделения нескольких строк в таблице сообщений (новое свойство Доступность выбора нескольких строк в таблице (MultiSelectAvailable) для экземпляра типа **Alarms**).
- Добавлена возможность скрытия/отображения кнопок квитирования всех сообщений (новое свойство Доступность кнопок квитирования всех сообщений (AckAllAvailable) для экземпляра типа **Alarms**).
- Количество предустановленных фильтров увеличено с 10 до 20 шт.
- Появилась возможность выбора папки экспорта с помощью диалога.

Изменения

- Свойство Доступность кнопок квитирования (AcknowledgmentAvailable) теперь называется Доступность функций квитирования (AcknowledgmentAvailable).

Исправления

- Исправлена ошибка, из-за которой в некоторых случаях уже проигранные звуки проигрывались повторно.
- Исправлены другие некритичные ошибки.

1.2.3

Новые возможности

Размер и положение главного окна приложения сохраняются в папке текущего пользователя:

- Windows: `C:\Users\<user>\Automiq]\HMI.Alarms\session.json`
- Linux: `/home/<user>/Automiq/HMI.Alarms/session.json`

Исправления

- Решены разного рода проблемы с отображением комментария при квитировании для сообщений, не требующих квитирования.
- Исправлена ошибка, из-за которой цвета темы не применялись для шрифта полей ввода даты и времени в интервале запроса.
- Решена проблема с сохранением файлов с настройками в папку для другого внешнего модуля, используемого в проекте (**Trends**).
- Исправлены другие некритичные ошибки.

Изменения документации

Редакция 2

Руководство пользователя

- Обновлено описание интеграции Alpha.HMI.Alarms в проект автоматизации (стр. 8).
- Добавлено описание обновления компонента Alpha.HMI.Alarms, встроенного в проект автоматизации.
- Актуализированы скриншоты.

Справочное руководство

- Изменено имя свойства UseAlphaSecurity на UseSecurity.

Редакция 3

Руководство пользователя

- Переработано описание интеграции Alpha.HMI.Alarms в проект Alpha.HMI: (стр. 8)
  - Раздел "Администрирование" заменен на раздел "Встраивание в проект".
  - Описание подключения внешнего модуля Alpha.HMI.Alarms и обновления встроенного модуля объединено в единый раздел (стр. 8).
  - Раздел "Установка и удаление" вынесен за пределы бывшего раздела "Администрирование".
  - Описано добавление экземпляра типа **Alarms** в проект (стр. 10).
  - Описание подключения к источнику данных включено в раздел с описанием добавления экземпляра типа **Alarms** в проект.
- Добавлено описание запуска Alpha.HMI.Alarms в виде самостоятельного приложения (стр. 12).

1.0

1.0.1

Улучшения

- добавлены столбцы **Группа важности** и **Активность** **(стр. 16)**
- в окне **Экспорт данных** теперь отображается статус экспорта

- в окне **Печать** добавлены стрелки для перехода по страницам

Исправленные ошибки

- при запуске с Alpha.HMI.WebViewer:
  - в окне **Печать** названия некоторых кнопок отображались не полностью
  - в окне **Экспорт данных** при успешном экспорте данных генерировалась ошибка
  - в окне **Параметры** в узле **Настройка событий** некорректно отображались названия столбцов
  - после применения любого фильтра пользователя сообщения переставали отображаться
  - не проигрывались звуки из-за некорректного построения пути к звуковым файлам
- в ОС Linux:
  - исправлено ошибочное накопление очереди звуковых алармов при длительной работе приложения
  - уменьшена длительность подготовки таблицы событий к печати
  - в оперативном режиме при задании фильтра отображения по столбцу **Важность** возникала ошибка
  - при использовании функции SetHistoricalInterval в строке запроса не отображался запрашиваемый временной интервал

1.0.2

Новые возможности

- кроме текущих форматов теперь список отображаемых сообщений о событиях можно сохранять в формате `.pdf`
- появилась возможность настройки начертания шрифта текста в строках сообщений и заголовках столбцов
- появилась возможность выбора столбцов, которые будут мигать
- появилась возможность настройки отображения сообщений для отдельных подусловий генерации события
- теперь цвет фона и текста строки можно настраивать не только для всех квитированных событий, но и для квитированных событий с определенной важностью

Исправленные ошибки

- альтернативная конфигурация не загружалась и подменялась файлами по умолчанию при указании абсолютного пути
- файлы альтернативной конфигурации создавались по неверному пути
- файл экспорта сохранялся без настроек фильтрации, сортировки и декорирования с подусловием
- событие по сигналу типа float или double отображалось со значением сигнала «**0**» вместо реального значения сигнала
- при печати вместо значений сигналов отображались некорректные символы
- декорирование событий с определенным подусловием не применялось при печати
- при запуске с Alpha.HMI.WebViewer:
  - после перезагрузки страницы путь сохранения файлов менялся на путь по умолчанию
  - отсутствовала возможность исправления значений в полях настроек и полях окна **Регистрация пользователя**

1.0.3

Новые возможности

Добавлена функция GetSoundActivity, позволяющая следить за переходом очереди звуков из пустого состояния в непустое.

Исправленные ошибки

Функция ClearAcknowledgedEvents не удаляла из таблицы сообщения о событиях, не требующих квитирования.

1.0.4

Новые возможности

- добавлена возможность предпросмотра вида таблицы и изменения ширины столбцов перед экспортом
- теперь за очередью звуков можно следить с помощью ссылок на команды (OnPlayingSounds и OnFinishedPlayingSounds), которые вызываются при событиях начала и окончания проигрывания очереди звуков

Изменения

- удалена функция GetSoundActivity, позволявшая следить за очередью звуков

Улучшения

- теперь сообщения об отсутствии звуков в папке выводятся в **Журнал времени исполнения** (desktop) и в **Журнал приложений** (web-версия)
- функция ClearOperativeEventList теперь возвращает значение типа int4 с результатом своей работы
- функция ClearAcknowledgedEvents теперь возвращает значение типа int4 с результатом своей работы

Исправленные ошибки

- фильтрация с помощью функции SetAdvancedFilter не выполнялась по столбцам важности, группы важности и времени
- настроенные условия фильтра отображения/фильтра запроса сбрасывались после выбора команды **Без фильтра**
- не блокировалась кнопка **Вернуться к значениям по умолчанию** при настройках вида таблицы по умолчанию
- в печать попадали не только сообщения, существующие в таблице на момент открытия окна предварительного просмотра, но и остальные поступившие сообщения
- функция ClearOperativeEventList и ClearAcknowledgedEvents работали в режиме снимка

Изменения документации

Редакция 2

Руководство пользователя

- обновлены имена дистрибутивов (стр. 6)
- обновлены пути к звуковым файлам (стр. 24)
- описана возможность просмотра в таблице признака активности подусловия (стр. 16), по которому было сгенерировано событие
- описана возможность просмотра условий фильтра подписки (стр. 38), установленного администратором
- описана возможность указания дополнительных данных (стр. 46) (имя пользователя, дата печати, номер страницы), попадающих в печать
- добавлен раздел с историей изменения Alpha.HMI.Alarms и документации (стр. 57)

Справочное руководство

- обновлено описание функции ClearAcknowledgedEvents
- обновлено описание функции ClearOperativeEventList

Редакция 3

Справочное руководство

- удалено описание свойства **Не квитировать события, не требующие квитирования** (DontAcknowEventsIfNotRequiring)
- добавлено описание свойства **Отображать миллисекунды** (ShowMilliseconds)
- обновлено описание функции RequestRecentEvents:
  - изменено описание возможных значений параметра onlyNotAcked
  - исправлены ошибки в синтаксисе примера
  - добавлено примечание об отсутствии звуков подгружаемых сообщений
  - расширено описание функции
  - расширено описание параметра intervalDuration
- обновлено описание функции SetAdvancedFilter:
  - в таблицу с перечислением идентификаторов добавлен столбец **Тип данных в столбце**
  - в таблицу с перечислением операторов добавлен столбец с указанием разрешенных типов
  - обновлены примеры
  - удалено описание неиспользуемых операторов
  - добавлено описание операторов И, ИЛИ, НЕ
  - обновлены правила составления условий фильтрации
  - расширено описание функции

Программный комплекс Альфа платформа

Alpha.HMI.Alarms 3.3

Справочник API

Редакция	Соответствует версии ПО
1. Предварительная	3.3.0

© АО «Атомик Софт», 2015-2025. Все права защищены.

Авторские права на данный документ принадлежат АО «Атомик Софт». Копирование, перепечатка и публикация любой части или всего документа не допускается без письменного разрешения правообладателя.

Содержание

1. Alarms **7**
1.1. Свойства 7
Квитировать события повторным кликом по выделенной строке (AcknowledgmentDoubleClick) 7
Квитировать последовательность событий (AllowAckSequence) 7
Запрашивать комментарий при квитировании (RequestAcknowledgmentComment) 7
Разрешить добавлять комментарий (AllowAddComment) 9
Дописывать название АРМ в комментарий квитирования (WriteComputerNameToComment) 9
Доступность функций квитирования (AcknowledgmentAvailable) 9
Доступность функций очистки (ClearingCurrentEventsAvailable) 10
Доступность редактирования настроек (EditingSettingsAvailable) 11
Доступность сохранения данных в табличный файл (ExportDataAvailability) 11
Доступность фильтрации (FilterAvailable) 11
Доступность кнопок управления проигрыванием звуков (SoundPlaybackManagmentAvailable) 12
Доступность кнопок функции Снимок (SnapshotAvailable) 12
Доступность просмотра оперативных событий (CurrentEventsAvailable) 12
Доступность просмотра истории событий (EventHistoryAvailable) 13
Доступность управления печатью (PrintingAvailable) 13
Видимость команд сортировки (EventTableSortAvailability) 13
Разрешить действия в столбце источник события (AllowActionInSourceColumn) 14
Имя приложения с правами безопасности (SecurityApplication) 15
Запрашивать активные события при запуске (RequestEventsStart) 16
Принудительное слежение за последним событием (FollowLatestEvent) 16
Слежение за последним поступившим событием (LookLatestEvent) 17
Беззвучный режим включен (Mute) 17
Доступность файловой системы (FileSystemAccess) 18
Список активных условий (ActiveConditionsList) 18
Путь к папке с конфигурацией (ConfigurationFolder) 19
Использовать сервер безопасности для определения прав (UseSecurity) 19
Ссылка на источник АР (init_ApSource) 19
Тег сигнала очистки очереди звуков (ClearSoundQueryTag) 19
Показывать панель инструментов (DisplayToolbar) 21
Показывать строку состояния (DisplayStatePanel) 21
Скрыть заголовки столбцов (HideColumnTitle) 21
Перенос строк (LineTransfer) 21
Не отображать неактивные квитированные события (свойство устарело) (HideAckedEvents) 22
Фильтр подписки (SubscriptionAdvancedFilter) 22
Доступность сортировки по заголовкам столбцов таблицы (EnableSortingOnGridHeader) 22
Возможность изменения вида пользовательского интерфейса (GUIModificationAvailable) 23
Отображать миллисекунды (ShowMilliseconds) 23
Доступность выбора нескольких строк в таблице (MultiSelectAvailable) 23
Активный режим работы (оперативный или исторический) (ActiveMode) 24
Максимальное количество обычных событий (MaxVisibleOrdinaryEvents) 24
Режим отображения полосы горизонтальной прокрутки (HorizontalScrollBarPolicy) 24
Режим отображения полосы вертикальной прокрутки (VerticalScrollBarPolicy) 25
Разрешение использования блокировки и подавления (SuppressAndBlock) 25
1.2. Функции 25
AckAll 25
AckSelected 27
ClearAcknowledgedEvents 27
ClearOperativeEventList 29
ClearSoundQueue 29
LoadConfiguration 29
LogOff 31

LogOn 31
SetAdvancedFilter 32
SetSnapshotState 48
SetAdvancedFilterAsJson 49
CancelHistoryQuery 58
ReloadHistory 58
SetAcknowledgeAvailability 58
SetAdvancedRequestFilterAsJson 60
SetHistoricalInterval 63
SetSize 64
SkipCurrentSound 64
RequestRecentEvents 65

2. Alarms_Table 66
2.1. Свойства 66
2.1.1. Ссылка на источник АР (init_ApSource) 66
2.1.2. Фильтр подписки (SubscriptionAdvancedFilter) 66
2.1.3. Активный режим работы (0-оперативный, 1 - исторический) (ActiveMode) 66
2.1.4. OnPrint 66
2.1.5. OnSave 66
2.1.6. OnLoad 66
2.1.7. OnAck 67
2.1.8. OnMouseRightRelease 67
2.1.9. OnHistoryLoad 67
2.1.10. OnOperativeLoad 67
2.1.11. OnError 67
2.1.12. OnActionInSourceColumn 67
2.1.13. OnExport 67
2.1.14. Параметры вызова экранной клавиатуры (pScreenKeyboard) 68
2.1.15. Разрешения пользователя (pPermissions) 68
2.1.16. Тема оформления (init_Theme) 68
2.1.17. Шаблоны (pTemplates) 68
2.1.18. Статистика (pEventStat) 68
2.1.19. OnSettingsChanged 68
2.1.20. pConfigTable 68
2.1.21. pConfigAcknowledge 68
2.1.22. pConfigPresetFilters 68
2.1.23. pConfigEvents 69
2.1.24. pConfigExport 69
2.1.25. pConfigSort 69
2.1.26. pConfigSession 69
2.1.27. pConfigSupression 69
2.1.28. pConfigViewFilter 69
2.1.29. pConfigViewFilter 69
2.1.30. pConfigPrint 69
2.1.31. pConfigExportView 69
2.1.32. pConfigToolbar 70
2.1.33. pConfigMenu 70
2.1.34. pConfigCommentPattern 70
2.1.35. pConfigSounds 70
2.1.36. pConfigPrintView 70
2.2. Функции 70
AckSelected 70
AckAll 70
AckExt 70
2.2.1. AddComment 71

2.2.2. ClearOperativeEventList 71
2.2.3. ClearAcknowledgedEvents 72
SetAdvancedRequestFilterAsJson 72
SetHistoricalInterval 72
RequestRecentEvents 72
CancelHistoryQuery 72
ReloadHistory 72
SaveToFile 72
SaveToFolder 73
LoadFromFile 73
LoadFromFolder 73
2.2.4. GetSelectedSource 74
SetSnapshotState 74
LoadConfiguration 74
SaveConfiguration 74
SetAcknowledgeAvailability 75

3. Alarms_Sounds 76
3.1. Свойства 76
3.1.1. ApSource 76
3.1.2. ClearSoundQueryTag 76
3.1.3. SubscriptionAdvancedFilter 76
3.1.4. Volume 76
3.1.5. OnPlayingSounds 76
3.1.6. OnFinishedPlayingSounds 76
3.1.7. OnSoundPlayed 77
3.1.8. Настройки звуков (pSounds) 77
3.1.9. Настройки подключения (pConnection) 77
3.1.10. Настройки вида таблицы (pConfigTable) 77
3.2. Функции 77
ClearSoundQueue 77
ReloadSoundQueue 77
SkipCurrentSound 77

4. EventStatistics 78
4.1. Свойства 78
Включен подсчёт количества по важности (Enabled) 78
Количество отображаемых (CountView) 78
Количество загруженных (CountTotal) 78
Количество по важности (CountViewBySeverity) 78
Имя открытого файла (FileName) 79
Текст ошибки (ErrorString) 79

5. Permissions 80
5.1. Свойства 80
5.1.1. Имя приложения с правами безопасности (SecurityApplication) 80
5.1.2. Использовать сервер безопасности для определения прав (UseSecurity) 80
5.1.3. Доступность редактирования настроек (EditingSettingsAvailable) 80
5.1.4. Доступность сохранения данных в табличный файл (ExportDataAvailability) 80
5.1.5. Доступность фильтрации (FilterAvailable) 80
5.1.6. Доступность просмотра оперативных событий (CurrentEventsAvailable) 80
5.1.7. Доступность просмотра истории событий (EventHistoryAvailable) 80
5.1.8. Доступность управления печатью (PrintingAvailable) 80
5.1.9. Видимость команд сортировки (EventTableSortAvailability) 81
5.1.10. Доступность файловой системы (FileSystemAccess) 81
5.1.11. Доступность функций квитирования (AcknowledgmentAvailable) 81

5.1.12. Доступность функций квитирования (AcknowledgmentAvailable) 81
5.1.13. Доступность кнопок управления проигрыванием звуков (SoundPlaybackManagmentAvailable) 81
5.1.14. Доступность кнопок функции Снимок (SnapshotAvailable) 81
5.1.15. Разрешение использования блокировки и подавления (SuppressionAvailable) 81
Доступность выбора принтера (PrinterSelectingAvailable) 81
Перечень допускаемых принтеров (AllowedPrinters) 82
5.1.16. Разрешить действия в столбце источник события (AllowActionInSourceColumn) 82
5.1.17. Возможность изменения вида пользовательского интерфейса (GUIModificationAvailable) 82
5.2. Функции 82
LogOn 82
LogOff 82

1. Alarms

1.1. Свойства

Квитировать события повторным кликом по выделенной строке (AcknowledgmentDoubleClick)

```
bool AcknowledgmentDoubleClick
```

Разрешает/запрещает квитирование событий двойным кликом.

Значение

«false»	Запретить квитирование событий двойным кликом.
«true»	Разрешить квитирование событий двойным кликом.

Квитировать последовательность событий (AllowAckSequence)

```
bool AllowAckSequence
```

Разрешает/запрещает квитирование последовательности событий (все отображаемые события одного источника событий).

Значение

«false»	Запретить квитирование последовательности событий.
«true»	Разрешить квитирование последовательности событий.

Запрашивать комментарий при квитировании (RequestAcknowledgmentComment)

```
bool RequestAcknowledgmentComment
```

Включает/отключает отображение окна для ввода комментария при каждом квитировании.

Значение

«false»	Отключить запрос комментария при квитировании.

«true»	Включить запрос комментария при квитировании.

Разрешить добавлять комментарий (AllowAddComment)

```
bool AllowAddComment
```

Разрешает/запрещает добавление комментариев для квитированных событий в оперативном и историческом режимах.

> ПРИМЕЧАНИЕ
> Рекомендуется активировать свойство только если в источниках данных присутствует Alpha.Server, поддерживающий функцию добавления комментариев.

Значение

«false»	Запретить добавление комментариев.
«true»	Разрешить добавление комментариев.

Дописывать название АРМ в комментарий квитирования (WriteComputerNameToComment)

```
bool WriteComputerNameToComment
```

Включает/выключает автоматическое добавление имени компьютера (с которого квитируется событие) в комментарий квитирования.

Значение

«0»	Не добавлять имя компьютера в комментарий.
«1»	Добавлять имя компьютера в комментарий.

Доступность функций квитирования (AcknowledgmentAvailable)

```
bool AcknowledgmentAvailable
```

С помощью свойства можно полностью заблокировать возможность квитирования или разрешить такую возможность.

Свойство влияет на:

- Доступ к кнопкам **Квитировать** и **Квитировать все** на панели инструментов.
- Доступ к командам **Квитировать**, **Квитировать все** и **Добавить комментарий** в контекстном меню (в историческом режиме только к команде **Добавить комментарий**).
- Возможность пользоваться функциями AckSelected и AckAll.

- Возможность квитировать двойным кликом по сообщению о событии.
- Возможность квитировать кнопкой в столбце **Квитировать**.

Значение

«false»	Возможность квитирования отключена.
«true»	Возможность квитирования включена.

Доступность функций очистки (ClearingCurrentEventsAvailable)

```
bool ClearingCurrentEventsAvailable
```

Отображает/скрывает кнопку **Очистить список** на панели инструментов и соответствующую команду в контекстном меню.

Значение

«false»	Скрыть кнопку и команду очистки списка.
«true»	Показать кнопку и команду очистки списка.

Доступность редактирования настроек (EditingSettingsAvailable)

```
bool EditingSettingsAvailable
```

Отображает/скрывает кнопку **Параметры** на панели инструментов и соответствующую команду в контекстном меню.

Значение

«false»	Запретить доступ к настройкам.
«true»	Разрешить доступ к настройкам.

Доступность сохранения данных в табличный файл (ExportDataAvailability)

```
bool ExportDataAvailability
```

Отображает/скрывает кнопку **Сохранить** на панели инструментов.

Значение

«false»	Скрыть кнопку сохранения списка событий в файл.
«true»	Показать кнопку сохранения списка событий в файл.

Доступность фильтрации (FilterAvailable)

```
bool FilterAvailable
```

Отображает/скрывает кнопку **Фильтр отображения** на панели инструментов и соответствующую команду в контекстном меню.

Значение

«false»	Скрыть кнопку фильтрации.
«true»	Показать кнопку фильтрации.

Доступность кнопок управления проигрыванием звуков (SoundPlaybackManagmentAvailable)

```
bool SoundPlaybackManagmentAvailable
```

Отображает/скрывает кнопки **Очистить очередь звуков**, **Пропустить звук**, **Включить проигрывание звуков** на панели инструментов и соответствующие команды в контекстном меню.

Значение

«false»	Скрыть кнопки и команды управления звуками.
«true»	Показать кнопки и команды управления звуками.

Доступность кнопок функции Снимок (SnapshotAvailable)

```
bool SnapshotAvailable
```

Отображает/скрывает кнопку **Снимок** на панели инструментов и соответствующую команду в контекстном меню.

Значение

«false»	Скрыть кнопку и команду снимка.
«true»	Показать кнопку и команду снимка.

Доступность просмотра оперативных событий (CurrentEventsAvailable)

```
bool CurrentEventsAvailable
```

Разрешает/запрещает просмотр оперативных событий. После включения свойства кнопка перехода в оперативный режим на панели инструментов и такая же команда в контекстном меню скрываются.

ОБРАТИТЕ ВНИМАНИЕ

Перед блокировкой просмотра оперативных событий заблаговременно переключитесь в исторический режим.

Значение

«true»	Разрешить просмотр оперативных событий.
«false»	Заблокировать просмотр оперативных событий.

Доступность просмотра истории событий (EventHistoryAvailable)

```
bool EventHistoryAvailable
```

Разрешает/запрещает просмотр истории событий. После включения свойства кнопка перехода в исторический режим на панели инструментов и такая же команда в контекстном меню скрываются

ОБРАТИТЕ ВНИМАНИЕ

Перед блокировкой просмотра истории событий заблаговременно переключитесь в оперативный режим.

Значение

«true»	Разрешить просмотр исторических событий.
«false»	Заблокировать просмотр исторических событий.

Доступность управления печатью (PrintingAvailable)

```
bool PrintingAvailable
```

Отображает/скрывает кнопку **Печать** на панели инструментов и соответствующую команду в контекстном меню.

Значения

«true»	Показать кнопку и команду отправки на печать.
«false»	Скрыть кнопку и команду отправки на печать.

Видимость команд сортировки (EventTableSortAvailability)

```
bool EventTableSortAvailability
```

Отображает/скрывает кнопку **Сортировка** на панели инструментов и такую же команду в контекстном меню.

Значение

«true»	Показать кнопку сортировки списка событий.
«false»	Скрыть кнопку сортировки списка событий.

Разрешить действия в столбце источник события (AllowActionInSourceColumn)

```
bool AllowActionInSourceColumn
```

Заполняет/не заполняет значение свойства objectID (стр. 1) идентификатором объекта события при нажатии на полный тег или относительный тег в столбце **Источник события** таблицы событий.

Значения

«true»	Заполнять значение свойства **objectID** идентификатором объекта события.
«false»	Не заполнять значение свойства **objectID** идентификатором объекта события.

Имя приложения с правами безопасности (SecurityApplication)

```
string SecurityApplication
```

Хранит имя приложения Alpha.Security, в котором содержится право **`FileSystemAccess`** (**`Доступность файловой системы`**), регулирующее доступность файловой системы для конкретного пользователя или группы пользователей.

Чтобы использовать сервер безопасности для определения прав текущего пользователя, установите значение «**`true`**» для свойства Использовать сервер безопасности для определения прав.

Примеры

ПРИМЕР

Указать имя приложения Alpha.HMI.Alarms.

```
Alarms_1.SecurityApplication = "Alarms";
```

Запрашивать активные события при запуске (RequestEventsStart)

```
bool RequestEventsStart
```

Активирует/деактивирует опрос источника данных о наличии активных событий при каждом подключении к серверу.

Значения

«true»	Запрашивать активные события с источника при каждом подключении.
«false»	Не запрашивать активные события с источника при каждом подключении.

Принудительное слежение за последним событием (FollowLatestEvent)

```
bool FollowLatestEvent
```

Включает/отключает постоянное отображение в поле видимости последнего добавленного события.

ПРИМЕЧАНИЕ

При включении свойства вы не сможете выделять и квитировать события в таблице. Для выделения строк и квитирования используйте режим снимка.

Значение

«true»	Следить за последним пришедшим событием.
«false»	Не следить за последним пришедшим событием.

Слежение за последним поступившим событием (LookLatestEvent)

```
bool LookLatestEvent
```

Включает/выключает перевод фокуса на каждое новое событие и автоматическую прокрутку таблицы сообщений до строки нового события, если выделено последнее поступившее событие.

Значение

«**true**»	Включить автоматическую прокрутку таблицы сообщений до строки нового события.
«**false**»	Отключить автоматическую прокрутку таблицы сообщений до строки нового события.

Беззвучный режим включен (Mute)

```
bool Mute
```

Включает/отключает все звуки.

Значение

«**true**»	Отключить звуки.
«**false**»	Включить звуки.

Доступность файловой системы (FileSystemAccess)

```
bool FileSystemAccess
```

Разрешает/запрещает доступ к файловой системе.

Если свойство не активно:

- любые команды импорта/экспорта вызывают специальные диалоговые окна, которые не содержат возможностей воздействия на файловую систему;
- отсутствует вкладка **Экспорт** в окне **Параметры**.

Значение

«true»	Разрешить доступ к файловой системе.
«false»	Ограничить доступ к файловой системе.

Список активных условий (ActiveConditionsList)

```
bool ActiveConditionsList
```

Устанавливает режим отображения таблицы сообщений: список активных условий или журнал.

Значение

«false»	Режим журнала.
«true»	Режим списка активных условий.

Путь к папке с конфигурацией (ConfigurationFolder)

```
string ConfigurationFolder
```

Полный или относительный (относительно папки `resources\AlarmSettings`) путь к папке сохранения/загрузки конфигурации.

Примеры

ПРИМЕР

Указать путь к папке `Alarms`, находящейся в папке проекта.

```
Alarms_1.ConfigurationFolder = "Alarms";
```

ПРИМЕР

Указать путь к папке `Alarms`, находящейся вне папки проекта.

```
Alarms_1.ConfigurationFolder = "C:\\Alarms";
```

Использовать сервер безопасности для определения прав (UseSecurity)

```
bool UseSecurity
```

Включает/выключает использование подсистемы безопасности Alpha.Security для определения прав пользователей.

Значения

«true»	Использовать Alpha.Security.
«false»	Не использовать Alpha.Security.

Ссылка на источник AP (init_ApSource)

Параметр инициализации. Используется для указания источника данных для экземпляра типа **`Alarms`**.

Тег сигнала очистки очереди звуков (ClearSoundQueryTag)

```
string ClearSoundQueryTag
```

Параметр инициализации. Используется для указания тега сигнала (тип bool) в DA сервере, значение **«true»** которого приводит к очистке очереди звуков.

Примеры

ПРИМЕР

Задать тег сигнала очистки очереди звуков.

```
Alarms_1.ClearSoundQueryTag = "Commands.Mute";
```

Показывать панель инструментов (DisplayToolbar)

```
bool DisplayToolbar
```

Отображает/скрывает панель инструментов.

Значение

«true»	Показать панель инструментов.
«false»	Скрыть панель инструментов.

Показывать строку состояния (DisplayStatePanel)

```
bool DisplayStatePanel
```

Отображает/скрывает строку состояния.

Значение

«true»	Показать строку состояния.
«false»	Скрыть строку состояния.

Скрыть заголовки столбцов (HideColumnTitle)

```
bool HideColumnTitle
```

Отображает/скрывает заголовки столбцов таблицы сообщений.

Значение

«true»	Скрыть заголовки столбцов.
«false»	Показать заголовки столбцов.

Перенос строк (LineTransfer)

```
bool LineTransfer
```

Включает/выключает перенос строк в таблице сообщений.

Значение

«true»	Включить перенос строк.
«false»	Отключить перенос строк.

Не отображать неактивные квитированные события (свойство устарело) (HideAckedEvents)

ОБРАТИТЕ ВНИМАНИЕ

Свойство устарело и будет удалено в следующих версиях. Рекомендуем вместо него использовать настройку **AckRequired** в Alpha.Server (модуль OPC AE Server).

```
bool HideAckedEvents
```

Отображает/скрывает неактивные квитированные события в таблице событий.

Значение

«true»	Скрыть неактивные квитированные события.
«false»	Показать неактивные квитированные события.

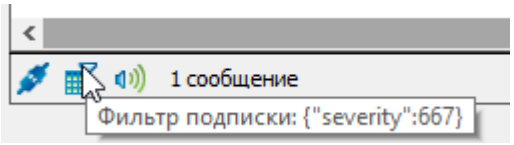
Фильтр подписки (SubscriptionAdvancedFilter)

```
string SubscriptionAdvancedFilter
```

Устанавливает фильтрацию (в формате JSON) для всех приходящих оперативных событий. Правила формирования условий фильтрации в формате JSON и примеры готовых фильтров аналогичны функции SetAdvancedFilterAsJson (стр. 49).

ПРИМЕЧАНИЕ

При включении фильтра подписки в статусной строке отображается индикатор.



Доступность сортировки по заголовкам столбцов таблицы (EnableSortingOnGridHeader)

```
bool EnableSortingOnGridHeader
```

Разрешает/запрещает сортировать по заголовкам столбцов таблицы.

Значение

«false»	Запрещена сортировка по заголовкам столбцов таблицы.
«true»	Разрешена сортировка по заголовкам столбцов таблицы.

Возможность изменения вида пользовательского интерфейса (GUIModificationAvailable)

```
bool GUIModificationAvailable
```

Разрешает/запрещает менять ширину столбцов мышкой прямо в рабочей области таблицы и отключать/включать отображение панели инструментов и статусной строки.

Значение

«false»	Запретить изменять вид пользовательского интерфейса.
«true»	Разрешить изменять вид пользовательского интерфейса.

Отображать миллисекунды (ShowMilliseconds)

```
bool ShowMilliseconds
```

Отображает/скрывает миллисекунды в столбцах с меткой времени: **Время генерации**, **Время срабатывания**, **Время деактивации** и **Время квитирования**.

Значение

«true»	Отображать миллисекунды.
«false»	Скрывать миллисекунды.

Доступность выбора нескольких строк в таблице (MultiSelectAvailable)

```
bool MultiSelectAvailable
```

Разрешает/запрещает одновременно выделять несколько строк в оперативном или историческом режиме.

Значение

«false»	Запрещено выделять несколько строк одновременно.
«true»	Разрешено выделять несколько строк одновременно.

Активный режим работы (оперативный или исторический) (ActiveMode)

```
int4 ActiveMode
```

Текущий режим работы.

Значение

«false»	Оперативный режим.
«true»	Исторический режим.

Максимальное количество обычных событий (MaxVisibleOrdinaryEvents)

```
uint4 MaxVisibleOrdinaryEvents
```

Максимальное количество строк журнала событий. Значение в пределах от **«50»** до **«100 000»**.

Режим отображения полосы горизонтальной прокрутки (HorizontalScrollBarPolicy)

```
uint1 HorizontalScrollBarPolicy
```

Управляет отображением горизонтальной полосы прокрутки.

Значение

«0»	Полоса отображается, когда содержимое не умещается в границах таблицы.
«1»	Полоса всегда скрыта.
«2»	Полоса всегда отображается.

Режим отображения полосы вертикальной прокрутки (VerticalScrollBarPolicy)

```
uint1 VerticalScrollBarPolicy
```

Управляет отображением вертикальной полосы прокрутки.

Значение

«0»	Полоса отображается, когда содержимое не умещается в границах таблицы.
«1»	Полоса всегда скрыта.
«2»	Полоса всегда отображается.

Разрешение использования блокировки и подавления (SuppressAndBlock)

```
bool SuppressAndBlock
```

Разрешает/запрещает выполнять блокировку и подавление событий. После включения свойства кнопка выполнения подавления и блокировки на панели инструментов, такая же команда в контекстном меню и команда изменения состояния подавления и блокировки скрываются.

Значение

«false»	Запретить выполнять блокировку и подавление событий.
«true»	Разрешить выполнять блокировку и подавление событий.

1.2. Функции

AckAll

```
int4 AckAll()
```

Квитирует все неквитированные события. Работает только в оперативном режиме.

Возвращаемое значение

«-2»	Пользователь не зарегистрирован или нет разрешения на квитирование по свойству **Доступность функций квитирования** или праву **`Acknowledgment`** в системе безопасности.
«-1»	Текущий режим работы - исторический.
«-4»	При успешном квитировании. Такое значение функция возвращает еще до факта квитирования, в момент открытия окна для ввода комментария.
«0»	В списке оперативных событий отсутствуют неквитированные события.

AckSelected

```
int4 AckSelected()
```

Квитирует выделенные в списке неквитированные события. Работает только в оперативном режиме.

Возвращаемое значение

«-2»	Пользователь не зарегистрирован или нет разрешения на выполнение операции по свойству **Доступность кнопок квитирования** или праву **Acknowledgment** в системе безопасности.
«-1»	Текущий режим работы - исторический.
«-3»	При успешном квитировании. Такое значение функция возвращает еще до факта квитирования, в момент открытия окна для ввода комментария.
«0»	Среди выделенных оперативных событий отсутствуют неквитированные события.

ClearAcknowledgedEvents

```
int4 ClearAcknowledgedEvents()
```

Удаляет из таблицы оперативного режима следующие сообщения:

- квитированные;
- не требующие квитирования (**AckRequired** = 0);
- группы важности **Прочие**, не требующие квитирования.

ПРИМЕЧАНИЕ

Функция недоступна, если:

- пользователь не зарегистрирован с помощью функции LogOn (стр. 31);
- для учетной записи пользователя не включено право очистки списка оперативных сообщений;
- для экземпляра типа возможность очистки списка сообщений запрещена администратором;
- включен режим снимка.

Возвращаемое значение

«-2»	У пользователя недостаточно прав для удаления сообщений.
«-1»	Сообщения не удалились, т.к. включен исторический режим или режим снимка.
«0»	Отсутствуют квитированные или не требующие квитирования события.
«N»	Число удаленных сообщений, значение > **«0»**.

ClearOperativeEventList

```
int4 ClearOperativeEventList()
```

Очищает список событий оперативного режима.

ПРИМЕЧАНИЕ

Функция недоступна, если:

- пользователь не зарегистрирован с помощью функции LogOn (стр. 31);
- для учетной записи пользователя не включено право очистки списка оперативных сообщений;
- для экземпляра типа возможность очистки списка сообщений запрещена администратором;
- включен режим снимка.

Возвращаемое значение

«-2»	У пользователя недостаточно прав для удаления сообщений.
«-1»	Сообщения не удалились, т.к. включен исторический режим или режим снимка.
«0»	В таблице нет сообщений.
«N»	Число удаленных сообщений, значение > **«0»**.

ClearSoundQueue

```
void ClearSoundQueue()
```

Очищает очередь звуков.

LoadConfiguration

```
bool LoadConfiguration (string configurationFolder)
```

Загружает и применяет указанную конфигурацию.

Параметры

configurationFolder	string	Абсолютный или относительный (относительно папки `resources\AlarmSettings`) путь к папке, в которой хранятся файлы конфигурации. Для загрузки конфигурации по умолчанию следует указать пустую строку ("").

Возвращаемое значение

«true»	Конфигурация загружена.
«false»	Не удалось загрузить конфигурацию. Вероятная причина - отсутствие указанной папки.

Примеры

ПРИМЕР

Загрузить конфигурацию по умолчанию.

```
Result: bool = Alarms_1.LoadConfiguration("");
  if (Result == 1){
    TextEdit.Text = "Конфигурация успешно загружена";
      }
  else{
    TextEdit.Text = "Конфигурация не загружена. Проверьте наличие папки";
      }
```

ПРИМЕР

Загрузить конфигурацию по абсолютному пути `C:\Folder1\Config`.

```
Result: bool = Alarms_1.LoadConfiguration("C:\\Folder1\\Config");
  if (Result == 1){
    TextEdit.Text = "Конфигурация успешно загружена";
      }
  else{
    TextEdit.Text = "Конфигурация не загружена. Проверьте наличие папки";
      }
```

ПРИМЕР

Загрузить конфигурацию по относительному пути `C:\WORK\HMI\Alarms_1.0\resources\AlarmSettings`.

```
Result: bool = Alarms_1.LoadConfiguration("Config");
  if (Result == 1){
    TextEdit.Text = "Конфигурация успешно загружена";
      }
  else{
    TextEdit.Text = "Конфигурация не загружена. Проверьте наличие папки";
      }
```

LogOff

```
bool LogOff()
```

Разрегистрирует текущего пользователя.

Возвращаемое значение

«1»	Пользователь успешно разрегистрирован.
«0»	Не удалось разрегистрировать пользователя.

LogOn

```
bool LogOn(string UserName)
```

Регистрирует указанного пользователя.

Параметры

UserName	string	имя пользователя

Возвращаемое значение

«1»	Пользователь успешно зарегистрирован.
«0»	Не удалось зарегистрировать пользователя.

SetAdvancedFilter

```
bool SetAdvancedFilter(string ColumnID, string FilterExpr)
```

Фильтрует все отображаемые сообщения о событиях по указанному столбцу и условию в виде логического выражения. Набор столбцов, по которым можно фильтровать с помощью SetAdvancedFilter, расширен по сравнению с фильтром пользователя (стр. 1), поэтому такая фильтрация считается расширенной.

Параметры

ПРИМЕЧАНИЕ

Оба параметра функции типа string, поэтому обязательно заключайте их в кавычки.

<table>
<tr><td>ColumnID</td><td>string</td><td>Идентификатор столбца таблицы сообщений.
За каждым столбцом закреплен собственный идентификатор, менять который нельзя.
<table>
<tr><th>столбец</th><th>идентификатор</th><th>тип данных в столбце</th></tr>
<tr><td>Время генерации</td><td>time</td><td>время</td></tr>
<tr><td>Время срабатывания</td><td>active_time</td><td>время</td></tr>
<tr><td>Время деактивации</td><td>deactive_time</td><td>время</td></tr>
<tr><td>Сообщение</td><td>message</td><td>строка</td></tr>
<tr><td>Источник события</td><td>source</td><td>строка</td></tr>
<tr><td>Важность</td><td>severity</td><td>число</td></tr>
<tr><td>Квитировано</td><td>ack</td><td>строка</td></tr>
<tr><td>Время квитирования</td><td>ack_time</td><td>время</td></tr>
<tr><td>Пользователь</td><td>actor_id</td><td>строка</td></tr>
<tr><td>Комментарий</td><td>ack_comment</td><td>строка</td></tr>
<tr><td>Активность</td><td>active</td><td>строка</td></tr>
<tr><td>Квитировать</td><td>ack_required</td><td>строка</td></tr>
<tr><td>Качество сигнала</td><td>quality</td><td>число</td></tr>
<tr><td>Значение сигнала</td><td>value</td><td>число</td></tr>
<tr><td>Группа важности</td><td>group</td><td>строка</td></tr>
</table></td></tr>
<tr><td>FilterExpr</td><td>string</td><td>Условие фильтрации в виде логического выражения. Чтобы фильтрация сработала, вам нужно правильно составить условие фильтрации. Ниже описаны операторы, которые можно использовать в условиях фильтрации и правила, которые помогут вам составить работающее условие.</td></tr>
</table>

Возвращаемое значение

«true»	Расширенный фильтр отображения установлен.
« false»	Расширенный фильтр отображения не установлен, т.к. условие фильтрации сформировано неверно.

ПРИМЕР

Ниже показаны примеры, как вывести результат выполнения функции в текстовое поле и в журнал времени исполнения.

```
//Вывод результата в текстовое поле.
filter: string = "условие фильтрации";
column: string = "идентификатор столбца";
if (!Alarms_1.SetAdvancedFilter(column, filter))
Text_3 = ("Фильтр: {" + filter + "} для столбца с идентификатором " + column + " не
установлен");
else
Text_3 = ("Фильтр: {" + filter + "} для столбца с идентификатором " + column + "
установлен");
```

```
//Вывод результата в журнал времени исполнения.
if (!Alarms_1.SetAdvancedFilter("идентификатор", "условие фильтрации"))
DebugTool_1.Log("Не удалось установить фильтр пользователя для идентификатор: условие
фильтрации");
else
DebugTool_1.Log("Фильтр пользователя для идентификатор: условие фильтрации");
```

Операторы и правила формирования условий фильтрации

Условие фильтрации для столбца нужно записывать в виде логического выражения с использованием операторов сравнения и/или логических операторов. Такие выражения могут содержать всего одно условие фильтрации, либо быть сложными и включать сразу несколько условий.

Когда вы составляете логическое выражение, помните, что внутри каждого столбца содержатся данные определенного типа (стр. 32) (в столбце **Сообщение** - строковые данные, в столбце **Важность** - число и т.д.), поэтому подбирайте операторы, работающие с этим типом. Ниже для всех операторов сказано с какими столбцами они работают.

Операторы сравнения

Операторы сравнения нужны для сравнения значений в фильтруемом столбце с какими-либо произвольными значениями. Такие операторы вы можете использовать для фильтрации по столбцам, содержащим число (стр. 32) или время (стр. 32).

<	меньше
>	больше

<=	меньше или равно
>=	больше или равно
=	равно
<>, !=	не равно

Логические операторы

Логические операторы нужны для сложных выражений, когда несколько условий объединяются в одно условие, а также для создания отрицательного условия фильтрации.

Символьное написание	Кириллическое написание	Англоязычное написание	Описание	Для столбцов
&	И, и	AND, and	логическое "и"	Можно использовать для фильтрации по любому столбцу.
|	ИЛИ, или	OR, or	логическое "или"	Можно использовать для фильтрации по любому столбцу.
!	НЕ, не	NOT, not	логическое "не"	Можно использовать только для столбцов, содержащих строку (стр. 32).

Правила составления логических выражений

- Дата и время записывается в формате: «**`dd.mm.yyyy hh:mm:ss.zzz`**» (dd - день, mm - месяц, yyyy - год, hh - часы, mm - минуты, ss - секунды, zzz - миллисекунды). Вы можете фильтровать столбец по времени (стр. 32) вплоть до миллисекунд, а можете обрезать формат и фильтровать, например, только по дате («**`DD.MM.YYYY`**») или по дате и конкретной минуте («**`dd.mm.yyyy hh:mm`**»). (Примеры (стр. 45))
- Вместо классического синтаксиса операций сравнения «**`A < B`**» можете использовать короткую запись вида «**`< B`**» (оператор при этом может быть любой). (Примеры (стр. 42). Подразумевается, что левый операнд (A) всегда содержит данные из фильтруемого столбца. Такая запись неприменима для столбца со строками (стр. 32), т.к. для строк операторы сравнения не работают.

- Еще один вариант короткой записи: запись только операнда без оператора сравнения. Этот вариант записи используется при фильтрации по точному значению. Результат фильтрации в таком случае зависит от типа данных (стр. 32) в фильтруемом столбце:
  - Для столбца с числами (стр. 32) такая запись соответствует сравнению на равенство, т.е. для фильтрации столбца по точному значению вы можете не указывать оператор = (равно) и записать просто значение, такая запись тоже будет правильной. (Примеры (стр. 41))

> ПРИМЕР
> Вместо «**=12**» можно записать «**12**».

  - Для столбца с текстом (стр. 32) такая запись соответствует проверке на содержание конкретного слова, т.е. для фильтрации столбца по точному слову просто запишите нужное слово без каких-либо операторов. После фильтрации останутся все строки, содержащие указанное слово, в каком месте строки оно бы ни находилось. (Примеры (стр. 36))

> ПРИМЕР
> Вместо «***Авария***» можно записать «**Авария**».

  - Для столбца с временем (стр. 32) такая запись заменяется на определенный интервал и уже в рамках этого интервала происходит выборка значений. (Примеры (стр. 45))

  Интервал, который будет использоваться, зависит от формата, в котором вы указали время:

Формат времени	Интервал
21.02.2018 10:50:06.123	>=21.02.2018 10:50:06.123 И <21.02.2018 10:50:06.124 Интервал, в пределах которого выполняется выборка значение, равен 1 миллисекунде.
21.02.2018 10:50:06 Не указываем миллисекунды.	>=21.02.2018 10:50:06 И <21.02.2018 10:50:07 Интервал, в пределах которого выполняется выборка значений, равен 1 секунде.
21.02.2018 10:50 Не указываем миллисекунды и секунды.	>=21.02.2018 10:50 И <21.02.2018 10:51 Интервал, в пределах которого выполняется выборка значений, равен 1 минуте.
21.02.2018 Не указываем время.	>=21.02.2018 00:00 И <22.02.2018 00:00 Интервал, в пределах которого выполняется выборка значений, равен суткам.

- Если выражение получается сложным, используйте круглые скобки для группировки нескольких условий фильтра и расстановки приоритетов операций. Помните, что результат такого выражения вычисляется по обычным правилам алгебры логики: в первую очередь вычисляются выражения внутри скобок, затем операции отрицания (NOT, !=, <>), затем умножения (AND) и в последнюю очередь - сложение (OR). (Пример (стр. 43))

- Для столбцов с текстом (стр. 32):
  - Условия фильтрации для столбцов с текстом можно записывать без учета регистра.
  - Чтобы записать условие фильтрации, при котором столбец с текстом фильтруется по нескольким находящимся не рядом словам, используйте оператор AND (И, и, &). Вы можете не указывать оператор AND (И, и, &), а поставить пробел между словами, такая запись тоже будет правильной. (Пример (стр. 37))
  - Чтобы записать условие фильтрации, при котором столбец с текстом фильтруется по неразрывной фразе, заключите нужную фразу в двойные кавычки.

  ПРИМЕЧАНИЕ
  В языке ОМ экранируйте двойные кавычки символом \. В языке JavaScript этого делать не нужно.

  После фильтрации останутся все строки, содержащие целиком указанную фразу, в каком месте строки она бы ни находилась. (Пример (стр. 38))

Фильтрация по столбцу с текстом

По одному слову

ПРИМЕР
Показать сообщения о событиях, квитированные пользователем Operator_1.

Условие фильтрации нужно применить к столбцу **Пользователь**, поэтому в первом параметре функции укажите идентификатор столбца - actor_id. В условии фильтрации (второй параметр функции) укажите одно слово: Operator_1.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("actor_id", "Operator_1");
```

ПРИМЕР
Показать сообщения о событиях, в тексте которых есть слово **«выше»** или слово **«превышает»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с оператором ИЛИ:

выше ИЛИ превышает.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("message", "выше ИЛИ превышает");
```

ПРИМЕР

Показать сообщения о событиях, в тексте которых есть слово **«Failed»**, или слово **«ниже»**, или слово **«close»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с двумя операторами ИЛИ:

Failed ИЛИ ниже ИЛИ close.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("message", "Failed ИЛИ ниже ИЛИ close");
```

ПРИМЕР

Показать только квитированные сообщения о событиях.

Условие фильтрации нужно применить к столбцу **Квитировано**, поэтому в первом параметре функции укажите идентификатор столбца - ack. В условии фильтрации (второй параметр функции) укажите одно слово: Да.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("ack", "Да");
```

По нескольким словам

ПРИМЕР

Показать сообщения о событиях, в тексте которых есть слова **«выше»**, **«аварийного»** и **«на»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с двумя операторами &, либо совсем без операторов:

выше & аварийного & на.

В итоге, параметры функции выглядят так:

```
//Первый вариант.
Alarms_1.SetAdvancedFilter("message", "выше & аварийного & на");
```

```
//Второй вариант.
Alarms_1.SetAdvancedFilter("message", "выше аварийного на");
```

ПРИМЕР

Показать сообщения о событиях, в комментариях к которым есть слова **«IGNATIEV»** и **«отправлено»**.

Условие фильтрации нужно применить к столбцу **Комментарий**, поэтому в первом параметре функции укажите идентификатор столбца - ack_comment. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с оператором И или совсем без операторов:

IGNATIEV И отправлено.

В итоге, параметры функции выглядят так:

```
//Первый вариант.
Alarms_1.SetAdvancedFilter("ack_comment", "IGNATIEV И отправлено");
```

```
//Второй вариант.
Alarms_1.SetAdvancedFilter("ack_comment", "IGNATIEV отправлено");
```

По точной фразе

ПРИМЕЧАНИЕ

В языке OM для точных фраз не забывайте экранировать двойные кавычки символом \.

ПРИМЕР

Показать сообщения о событиях, содержащие точную фразу **«выше верхнего»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message. Само условие фильтрации (второй параметр функции) заключите в кавычки:

\"\"выше верхнего\"\" (язык Alpha.Om) или ""выше верхнего"" (язык JavaScript).

В итоге, параметры функции выглядят так:

```
//язык OM
Alarms_1.SetAdvancedFilter("message", "\"\"выше верхнего\"\"");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilter("message", '""выше верхнего""');
```

ПРИМЕР

Среди сообщений о событиях, в тексте которых есть точная фраза **«по уровню»** или подстрока **«уровн»**, показать только те сообщения, в которых есть слово **«ниже»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message.

Само условие фильтрации (второй параметр функции) составьте так:

1. Чтобы выбрать сообщения со словосочетанием **«по уровню»** или подстрокой **«уровн»**, запишите в скобках выражение с оператором или. Точную фразу **«по уровню»** заключите в кавычки:

```
(\"\"по уровню\"\" или уровн)
```

2. Чтобы из полученных сообщений показать только те сообщения, которые содержат слово **«ниже»**, пропишите после скобок это слово с оператором и:

```
(\"\"по уровню\"\" или уровн) и ниже
```

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("message", "(\"\"по уровню\"\" или уровн) и ниже");
```

ПРИМЕР

Среди сообщений о событиях, в тексте которых есть точная фраза **«ниже аварийного»** или слово **«опустилось»** показать только те сообщения, в которых есть точная фраза **«датчик 1»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message.

Само условие фильтрации (второй параметр функции) составьте так:

1. Чтобы выбрать сообщения сточной фразой **«ниже аварийного»** или словом **«опустилось»**, запишите в скобках выражение с оператором или. Точную фразу **«ниже аварийного»** заключите в кавычки:

```
(\"\"ниже аварийного\"\" или опустилось)
```

2. Чтобы из полученных сообщений показать только те сообщения, которые содержат точную фразу **«датчик 1»**, пропишите после скобок эту фразу с оператором и. Точную фразу **«датчик 1»** заключите в кавычки:

```
(\"\"ниже аварийного\"\" или опустилось) и \"\"датчик 1\"\"")
```

В итоге, параметры функции выглядят так:

```
//язык ОМ
Alarms_1.SetAdvancedFilter("message", "(\"\"ниже аварийного\"\" или опустилось) и
\"\"датчик 1\"\"");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilter("message", '(""ниже аварийного"" или опустилось) и
""датчик 1""');
```

Исключение отдельных слов

ПРИМЕР

Показать сообщения о событиях, в названиях групп важности которых нет подстроки **«inf»**.

Условие фильтрации нужно применить к столбцу **Група важности**, поэтому в первом параметре функции укажите идентификатор столбца - group. В условии фильтрации (второй параметр функции) укажите подстроку **«inf»** с оператором NOT:

NOT inf.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("group", "NOT inf");
```

ПРИМЕР

Показать сообщения о событиях, в тексте которых нет слова **«предел»**, подстроки **«уров»**, и подстроки **«пункт25»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message. Для составления самого условия фильтрации (второй параметр функции) вы можете:

- Заключить в скобки все указанные подстроки и установить перед скобками один оператор НЕ:

  НЕ(уров предел пункт25).

- Либо можете поставить оператор НЕ перед каждой подстрокой, а также поставить оператор или между подстроками:

  НЕ уров или НЕ предел или НЕ пункт25.

В итоге, параметры функции выглядят так:

```
//Первый вариант.
Alarms_1.SetAdvancedFilter("message", "НЕ(уров предел пункт25)");
```

```
//Второй вариант.
Alarms_1.SetAdvancedFilter("message", "НЕ уров или НЕ предел или НЕ пункт25");
```

Фильтрация по столбцу с числом

По точному значению

ПРИМЕР

Показать сообщения о событиях, важность которых равна **«660»**.

Условие фильтрации нужно применить к столбцу **Важность**, поэтому в первом параметре функции укажите идентификатор столбца - severity. В условии фильтрации (второй параметр функции) укажите число **«660»** с оператором = или без него.

В итоге, параметры функции выглядят так:

```
//Первый вариант.
Alarms_1.SetAdvancedFilter("severity", "=660");
```

```
//Второй вариант.
Alarms_1.SetAdvancedFilter("severity", "660");
```

ПРИМЕР

Показать сообщения о событиях, значение сигнала в которых равно **«-58»** или **«59»**.

Условие фильтрации нужно применить к столбцу **Значение сигнала**, поэтому в первом параметре функции укажите идентификатор столбца - value. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с операторами | и =:

=-58 | =59.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("value", "=-58 | =59");
```

По диапазону значений

ПРИМЕР

Показать сообщения о событиях, важность которых больше или равна **«9»**.

Условие фильтрации нужно применить к столбцу **Важность**, поэтому в первом параметре функции укажите идентификатор столбца - severity. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с оператором >=:

>=9.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("severity", ">=9");
```

ПРИМЕР

Показать сообщения о событиях, важность которых больше или равна **«40»** и меньше **«100»**.

Условие фильтрации нужно применить к столбцу **Важность**, поэтому в первом параметре функции укажите идентификатор столбца - severity. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с операторами И, >= и <:

>=40 И <100.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("value", ">=40 И <100");
```

ПРИМЕР

Показать сообщения о событиях с важностью от **«1»** до **«300»** или **«1000»**.

Условие фильтрации нужно применить к столбцу **Важность**, поэтому в первом параметре функции укажите идентификатор столбца - severity.

Само условие фильтрации (второй параметр функции) составьте так:

1. Чтобы выбрать сообщения с важностью от **«1»** до **«300»**, запишите в скобках выражение с операторами AND, >= и <=:

```
(>=1 AND <=300)
```

2. Чтобы к полученным сообщениям добавить еще сообщения с важностью **«1000»**, запишите после скобок это число с операторами OR и =:

```
(>=1 AND <=300) OR =1000"
```

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("severity", "(>=1 AND <=300) OR =1000");
```

ПРИМЕР

Показать сообщения о событиях, в которых значение сигнала находится в диапазоне от **«5»** до **«80»** и не равно **«50»**.

Условие фильтрации нужно применить к столбцу **Значение сигнала**, поэтому в первом параметре функции укажите идентификатор столбца - value.

Само условие фильтрации (второй параметр функции) составьте так:

1. Чтобы выбрать сообщения о событиях, содержащие значения сигнала в диапазоне от **«5»** до **«80»**, запишите в скобках выражение с операторами и, >= и <=:

```
(>=5 и <=80)
```

2. Чтобы среди полученных сообщений отбросить сообщения, в которых значение равно **«50»**, запишите после скобок это число с операторами и и !=:

```
(>=5 и <=80) и !=50
```

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("value", "(>=5 и <80) и !=50");
```

Исключение отдельных значений

Чтобы записать условие, при котором из столбца с числами исключатся определенные значения, используйте оператор !=, <> (не равно).

ПРИМЕР

Не показывать сообщения о событиях, важность которых равна **«667»**.

Условие фильтрации нужно применить к столбцу **Важность**, поэтому в первом параметре функции укажите идентификатор столбца - severity. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с оператором !=:

!=667.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("severity", "!=667");
```

ПРИМЕР

Не показывать сообщения о событиях, качество значения сигнала в которых равно **«0»** или **«216»**.

Условие фильтрации нужно применить к столбцу **Качество сигнала**, поэтому в первом параметре функции укажите идентификатор столбца - quality. Для составления самого условия фильтрации (второй параметр функции) вы можете:

- Заключить в скобки указанные значения качества с оператором or и установить перед скобками один оператор <>:

  <>(216 or 0).

- Либо можете поставить оператор <> перед каждым значением качества, а также поставить оператор and между значениями:

  <>216 and <>0.

В итоге, параметры функции выглядят так:

```
//Первый вариант.
Alarms_1.SetAdvancedFilter("quality", "<>(216 or 0)");
```

```
//Второй вариант.
Alarms_1.SetAdvancedFilter("quality", "<>216 and <>0");
```

Фильтрация по столбцу с временем

По точному значению

ПРИМЕР

Показать сообщения о событиях, возникших **«26.01.2022»**.

Условие фильтрации нужно применить к столбцу **Время срабатывания**, поэтому в первом параметре функции укажите идентификатор столбца - active_time. В условии фильтрации (второй параметр функции) укажите нужную дату.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("active_time", "26.01.2022");
```

ПРИМЕР

Показать сообщения о событиях, возникших **«28.11.2022 10:34:03»** или **«16.11.2022»**.

Условие фильтрации нужно применить к столбцу **Время срабатывания**, поэтому в первом параметре функции укажите идентификатор столбца - active_time. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с оператором ИЛИ:

28.11.2022 10:34:03 ИЛИ 16.11.2022.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("active_time", "28.11.2022 10:34:03 ИЛИ 16.11.2022");
```

По интервалу

ПРИМЕР

Показать сообщения о событиях, деактивированных раньше, чем **«16.08.2022 09:20:37»**.

Условие фильтрации нужно применить к столбцу **Время деактивации**, поэтому в первом параметре функции укажите идентификатор столбца - deactive_time. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с оператором <.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("deactive_time", "<16.08.2022 09:20:37");
```

ПРИМЕР

Показать сообщения о событиях, время активации условия которых находится в промежутке между **«25.01.2022 11:50:00»** и **«26.01.2022 15:50:00»**.

Условие фильтрации нужно применить к столбцу **Время генерации**, поэтому в первом параметре функции укажите идентификатор столбца - time. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с операторами >=, <= и AND.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("time", ">=25.01.2022 11:50:00 AND <=26.01.2022
15:50:00");
```

ПРИМЕР

Показать сообщения о событиях, время активации условия которых находится в промежутке между **«28.11.2022 10:33:00»** и **«28.11.2022 10:35:00»** или между **«16.11.2022 11:15:00»** и **«16.11.2022 11:15:33»**. Условие фильтрации нужно применить к столбцу **Время генерации**, поэтому в первом параметре функции укажите идентификатор столбца - time.

Само условие фильтрации (второй параметр функции) составьте так:

1. Чтобы выбрать сообщения о событиях, возникших в промежутке между **«28.11.2022 10:33:00»** и **«28.11.2022 10:35:00»**, запишите в скобках выражение с операторами AND, > и <:

```
(>28.11.2022 10:33:00 AND <28.11.2022 10:35:00)
```

2. Чтобы выбрать сообщения о событиях, возникших в промежутке между между **«16.11.2022 11:15:00»** и **«16.11.2022 11:15:33»**, запишите в скобках выражение с операторами AND, > и <:

```
(>16.11.2022 11:15:00 AND <16.11.2022 11:15:33)
```

3. Чтобы объединить два полученных выражения в одно условие, запишите между первым и вторым выражением оператор OR.

Вы можете записать условие фильтрации без использования скобок. В таком случае, с учетом приоритетов операций, результат будет тот же.

В итоге, параметры функции выглядят так:

```
//Первый вариант. Добавляем скобки, выделяющие диапазоны времени.
Alarms_1.SetAdvancedFilter("time", "(>28.11.2022 10:33:00 AND <28.11.2022 10:35:00)
OR (>16.11.2022 11:15:00 AND <16.11.2022 11:15:33)");
```

```
//Второй вариант без скобок, выделяющих диапазоны.
Alarms_1.SetAdvancedFilter("time", ">28.11.2022 10:33:00 AND <28.11.2022 10:35:00 OR
>16.11.2022 11:15:00 AND <16.11.2022 11:15:33");
```

ПРИМЕР

Показать сообщения о событиях, квитированных раньше, чем **«01.04.2022»** или позже, чем **«01.01.2023»**.

Условие фильтрации нужно применить к столбцу **Время квитирования**, поэтому в первом параметре функции укажите идентификатор столбца - ack_time. Само условие фильтрации (второй параметр функции) составьте в виде логического выражения с операторами >, < и или.

В итоге, параметры функции выглядят так:

```
Alarms_1.SetAdvancedFilter("ack_time", "<01.04.2022 или >01.01.2023");
```

Фильтрация по нескольким столбцам одновременно

Чтобы записать условие фильтрации по нескольким столбцам, вам нужно для каждого столбца прописать вызов функции SetAdvancedFilter отдельной строкой.

ПРИМЕР

Показать все сообщения о событиях, квитированные пользователем **«User_1»**.

Отдельной строкой запишите условие для столбца **Пользователь** (идентификатор actor_id), а следом, другой строкой запишите условие для столбца **Квитировано** (идентификатор ack):

```
DebugTool_1.Log(Alarms_1.SetAdvancedFilter("actor_id","User_1"));
DebugTool_1.Log(Alarms_1.SetAdvancedFilter("ack","Да"));
```

ПРИМЕР

Показать все сообщения о событиях, квитированные пользователем **«User 1»**, содержащие значение сигнала **«50»** и слово **«Принято»** в комментарии квитирования.

Отдельной строкой запишите условие для столбца **Пользователь** (идентификатор actor_id), далее отдельными строками запишите условия для столбцов Комментарий (идентификатор ack_comment) и **Значение сигнала** (идентификатор value):

```
DebugTool_1.Log(Alarms_1.SetAdvancedFilter("actor_id","\"\"User 1\"\""));
DebugTool_1.Log(Alarms_1.SetAdvancedFilter("ack_comment","Принято"));
DebugTool_1.Log(Alarms_1.SetAdvancedFilter("value","50"));
```

Отключение фильтрации по столбцу

Чтобы отключить фильтрацию по какому-либо столбцу, укажите идентификатор столбца и пустое условие фильтрации.

ПРИМЕР

Отключить фильтрацию по столбцу **Значение сигнала**.

```
Alarms_1.SetAdvancedFilter("value","");
```

SetSnapshotState

```
void SetSnapshotState(bool isOn)
```

Включает и отключает режим снимка.

Параметры

isOn	bool	Управление режимом снимка: › **«true»** - включить режим снимка; › **«false»** - отключить режим снимка.

Примеры

ПРИМЕР

Включить режим снимка

```
Alarms_1.SetSnapshotState(true);
```

SetAdvancedFilterAsJson

```
bool SetAdvancedFilterAsJson(string ColumnID, string FilterExpr)
```

Работает аналогично SetAdvancedFilter. Единственное отличие в том, что условие фильтрации задается в формате JSON.

Параметры

<table>
<tr><td>ColumnID</td><td>string</td><td>Идентификатор столбца таблицы сообщений.
За каждым столбцом закреплен собственный идентификатор, менять который нельзя.
<table>
<tr><th>столбец</th><th>идентификатор</th><th>тип данных в столбце</th></tr>
<tr><td>Время генерации</td><td>time</td><td>время</td></tr>
<tr><td>Время срабатывания</td><td>active_time</td><td>время</td></tr>
<tr><td>Время деактивации</td><td>deactive_time</td><td>время</td></tr>
<tr><td>Сообщение</td><td>message</td><td>строка</td></tr>
<tr><td>Источник события</td><td>source</td><td>строка</td></tr>
<tr><td>Важность</td><td>severity</td><td>число</td></tr>
<tr><td>Квитировано</td><td>ack</td><td>строка</td></tr>
<tr><td>Время квитирования</td><td>ack_time</td><td>время</td></tr>
<tr><td>Пользователь</td><td>actor_id</td><td>строка</td></tr>
<tr><td>Комментарий</td><td>ack_comment</td><td>строка</td></tr>
<tr><td>Активность</td><td>active</td><td>строка</td></tr>
<tr><td>Квитировать</td><td>ack_required</td><td>строка</td></tr>
<tr><td>Качество сигнала</td><td>quality</td><td>число</td></tr>
<tr><td>Значение сигнала</td><td>value</td><td>число</td></tr>
<tr><td>Группа важности</td><td>group</td><td>строка</td></tr>
</table></td></tr>
<tr><td>FilterExpr</td><td>string</td><td>Условие фильтрации в виде логического выражения в формате JSON. Чтобы фильтрация сработала, вам нужно правильно составить условие фильтрации. Ниже описаны операторы, которые можно использовать в условиях фильтрации и правила, которые помогут вам составить работающее условие.</td></tr>
</table>

Возвращаемое значение

«true»	Расширенный фильтр отображения установлен.
«false»	Расширенный фильтр отображения не установлен, т.к. условие фильтрации сформировано неверно.

ПРИМЕР

Ниже показаны примеры, как вывести результат выполнения функции в текстовое поле и в журнал времени исполнения.

```
//Вывод результата в текстовое поле.
filter: string = "условие фильтрации";
column: string = "идентификатор столбца";
if (!Alarms_1.SetAdvancedFilterAsJson(column, filter))
Text_3 = ("Фильтр: {" + filter + "} для столбца с идентификатором " + column + " не
установлен");
else
Text_3 = ("Фильтр: {" + filter + "} для столбца с идентификатором " + column + "
установлен");
```

```
//Вывод результата в журнал времени исполнения.
if (!Alarms_1.SetAdvancedFilterAsJson("идентификатор", "условие фильтрации"))
DebugTool_1.Log("Не удалось установить фильтр пользователя для идентификатор: условие
фильтрации");
else
DebugTool_1.Log("Фильтр пользователя для идентификатор: условие фильтрации");
```

Операторы и правила формирования условий фильтрации

Условие фильтрации для столбца нужно записывать в виде объекта JSON с использованием операторов сравнения и/или логических операторов. Такие выражения могут содержать всего одно условие фильтрации (одиночный объект JSON: {"ключ": "значение"}), либо быть сложными и включать сразу несколько условий (составной объект JSON, состоящий из нескольких объектов).

Когда вы составляете логическое выражение, помните, что внутри каждого столбца содержатся данные определенного типа (стр. 49) (в столбце **Сообщение** - строковые данные, в столбце **Важность** - число и т.д.), поэтому подбирайте операторы, работающие с этим типом. Ниже для всех операторов сказано с какими столбцами они работают.

Операторы сравнения

Операторы сравнения нужны для сравнения значений в фильтруемом столбце с какими-либо произвольными значениями. Такие операторы вы можете использовать для фильтрации по столбцам, содержащим число (стр. 49) или время (стр. 49).

<	меньше

>	больше
<=	меньше или равно
>=	больше или равно
==	равно
!=	не равно

Логические операторы

Логические операторы нужны для сложных выражений, когда несколько условий объединяются в одно условие, а также для создания отрицательного условия фильтрации.

Символьное написание	Англоязычное написание	Описание	Для столбцов
&	AND, and	логическое "и"	Можно использовать для фильтрации по любому столбцу.
|	OR, or	логическое "или"	Можно использовать для фильтрации по любому столбцу.
!	NOT, not	логическое "не"	Можно использовать только для столбцов, содержащих строку.

Логические выражения в формате JSON

Форматы составного объекта JSON

В зависимости от используемой операции составной объект JSON может иметь формат:

- {"and": [ARRAY]}
- {"or": [ARRAY]}
- { "not": { OBJECT } }
- { "операция сравнения": { VARIABLE } }

VARIABLE	Переменная. Имеет структуру: <имя переменной>: <значение переменной>. В имени переменной вы можете написать **«text»** и тогда в качестве значения такой переменной укажите значение, которое вы видите в таблице. Либо можете написать **«data»** и тогда в имени переменной укажите внутреннее значение, хранящееся в источнике данных. Например, для столбца **Квитировано** (ack) переменная **«text»** имеет значение **«Да»**, а переменная **«data»** - значение **«true»**.
OBJECT	Переменная или составной объект JSON любого из указанных выше форматов.
ARRAY	Массив. Может состоять из переменных и составных объектов JSON (любого формата) в различных комбинациях.

Правила составления логических выражений в формате JSON

- Операцию сравнения == можно не указывать. В таком случае объект будет иметь формат { VARIABLE }.
- В столбцах с датой и временем значение нужно указывать в формате UTC. Для начала переведите ваше строковое значение даты в метку времени формата uint8, а затем полученное значение переведите в формат UTC. Для перевода вы можете использовать функции Parse и ToUtc языка Alpha.Om.
- Для столбцов с текстом:
  - Условия фильтрации для столбцов с текстом можно записывать без учета регистра.
  - Чтобы записать условие фильтрации, при котором столбец с текстом фильтруется по нескольким находящимся не рядом словам, используйте оператор and (&).
- Условие фильтрации записывается в виде строки в кавычках, поэтому для правильной работы фильтра в языке OM экранируйте все остальные кавычки в строке символом \. В языке JavaScript этого делать не нужно.

Примеры

Фильтрация по столбцу с текстом

ПРИМЕР

Показать сообщения о событиях, в тексте которых есть слово **«поставлен»** или слово **«снят»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message. Само условие фильтрации (второй параметр функции) составьте в виде составного объекта JSON с оператором or.

Формат составного объекта JSON в данном случае будет таким: {"or": [ARRAY]}, где ARRAY (массив) состоит из двух VARIABLE (переменных): data: поставлен и data:снят.

В итоге, параметры функции выглядят так:

```
//язык OM
Alarms_1.SetAdvancedFilterAsJson("message","{\"or\":[{\"data\":\"*поставлен*\"},
{\"data\":\"*снят*\"}]}");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilterAsJson("message",'{"or":[{"data":"*поставлен*"},
{"data":"*снят*"}]}');
```

```
//язык OM (выводим результат выполнения функции в текстовое поле)
Filter: string = "{\"or\":[{\"data\":\"*поставлен*\"},{\"data\":\"*снят*\"}]}";
Result: bool = Alarms_1.SetAdvancedFilterAsJson("message", Filter);
TextEdit_1.Text = Str.ToString(Result);
```

ПРИМЕР

Показать сообщения о событиях, содержащие точную фразу **«выше аварийного»**.

Условие фильтрации нужно применить к столбцу **Сообщение**, поэтому в первом параметре функции укажите идентификатор столбца - message. Само условие фильтрации составьте по формату { "операция сравнения": { VARIABLE } }, где VARIABLE (переменная) - data: выше аварийного. Операцию сравнения в данном случае можно не ставить.

В итоге, параметры функции выглядят так:

```
//язык OM
Alarms_1.SetAdvancedFilterAsJson("message","{\"data\":\"*выше аварийного*\"}");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilterAsJson("message",'{"data":"*выше аварийного*"}');
```

ПРИМЕР

Показать сообщения о событиях, в комментариях к которым есть слова **«IGNATIEV»** и **«отправлено»**.

Условие фильтрации нужно применить к столбцу **Комментарий**, поэтому в первом параметре функции укажите идентификатор столбца - ack_comment. Само условие фильтрации (второй параметр функции) составьте в виде составного объекта JSON с оператором AND.

Формат составного объекта JSON в данном случае будет таким: {"and": [ARRAY]}, где ARRAY (массив) состоит из двух VARIABLE (переменных): data: IGNATIEV и data:отправлено.

В итоге, параметры функции выглядят так:

```
//язык OM
Alarms_1.SetAdvancedFilterAsJson("ack_comment","{\"AND\":[{\"data\":\"*IGNATIEV*\"},
{\"data\":\"*отправлено*\"}]}");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilterAsJson("ack_comment",'{"AND":[{"data":"*IGNATIEV*"},
{"data":"*отправлено*"}]}');
```

ПРИМЕР

Показать только квитированные сообщения о событиях.

Условие фильтрации нужно применить к столбцу **Квитировано**, поэтому в первом параметре функции укажите идентификатор столбца - ack. Само условие фильтрации (второй параметр функции) составьте по формату { "операция сравнения": { VARIABLE } }, где VARIABLE (переменная) - text: Да. Операцию сравнения в данном случае можно не ставить.

В итоге, параметры функции выглядят так:

```
//язык ОМ
Alarms_1.SetAdvancedFilterAsJson("ack", "{\"text\":\"Да\"}");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilterAsJson("ack", '{"text":"Да"}');
```

Фильтрация по столбцу с числом

ПРИМЕР

Показать сообщения о событиях, важность которых равна **«100»**.

Условие фильтрации нужно применить к столбцу **Важность**, поэтому в первом параметре функции укажите идентификатор столбца - severity. Само условие фильтрации (второй параметр функции) составьте по формату { "операция сравнения": { VARIABLE } }, где VARIABLE (переменная) - data: 100. Операцию сравнения в данном случае можно не ставить.

В итоге, параметры функции выглядят так:

```
//язык ОМ
Alarms_1.SetAdvancedFilterAsJson("severity","{\"data\":100}");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilterAsJson("severity",'{"data":100}');
```

ПРИМЕР

Показать сообщения о событиях, важность которых больше или равна **«100»** и меньше или равна **«667»**.

Условие фильтрации нужно применить к столбцу **Важность**, поэтому в первом параметре функции укажите идентификатор столбца - severity. Само условие фильтрации (второй параметр функции) составьте в виде составного объекта JSON с оператором and.

Формат составного объекта JSON в данном случае будет таким: {"and": [ARRAY]}, где ARRAY (массив) состоит из двух конструкций формата { "операция сравнения": { VARIABLE } }: >={data:100} и <= {data:667}.

В итоге, параметры функции выглядят так:

```
//язык ОМ
Alarms_1.SetAdvancedFilterAsJson("severity","{\"and\":[{\">=\":{\"data\":100}},
{\"<=\":{\"data\":667}}]}");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilterAsJson("severity",'{"and":[{">=":{"data":100}},{"<=":
{"data":667}}]}');
```

ПРИМЕР

Показать сообщения о событиях, значение сигнала в которых равно **«-58»** или **«59»**.

Условие фильтрации нужно применить к столбцу **Значение сигнала**, поэтому в первом параметре функции укажите идентификатор столбца - value. Само условие фильтрации (второй параметр функции) составьте в виде составного объекта JSON с оператором or.

Формат составного объекта JSON в данном случае будет таким: {"or": [ARRAY]}, где ARRAY (массив) состоит из двух VARIABLE (переменных): data: -58 и data:-59.

В итоге, параметры функции выглядят так:

```
//язык ОМ
Alarms_1.SetAdvancedFilterAsJson("value","{\"or\":[{\"data\":-58},{\"data\":-59}]}");
```

```
//язык JavaScript
Alarms_1.SetAdvancedFilterAsJson("value",'{"or":[{"data":-58},{"data":-59}]}');
```

Фильтрация по столбцу с временем

ПРИМЕР

Показать сообщения о событиях, возникших **«09.02.2024»**.

Условие фильтрации нужно применить к столбцу **Время срабатывания**, поэтому в первом параметре функции укажите идентификатор столбца - active_time.

Для удобства работы с датой **«09.02.2024»** её нужно представить в таком виде: >= 09.02.2024 00:00:00.000 И < 10.02.2024 00:00:00.000. Далее две полученные даты нужно перевести в формат UTC и преобразовать в строки, чтобы была возможность записать их во второй строковый параметр функции.

Чтобы сформировать условие фильтрации (второй параметр функции), выполните:

1. Переведите первую дату **«09.02.2024»** в формат UTC с помощью функций Parse и ToUtc языка Alpha.Om.

```
date1:uint8 = DateTime.Parse("09.02.2024");
date1UTC: uint8 = DateTime.ToUtc(date1);
```

2. Преобразуйте полученное значение в строку.

```
stringDate1UTC:string = String.ToString(date1UTC);
```

3. Переведите первую дату **«10.02.2024»** в формат UTC с помощью функций Parse и ToUtc языка Alpha.Om.

```
date2:uint8 = DateTime.Parse("10.02.2024");
date2UTC: uint8 = DateTime.ToUtc(date2);
```

4. Преобразуйте полученное значение в строку.

```
stringDate2UTC:string = String.ToString(date2UTC);
```

5. Сформируйте составной объект JSON. Формат составного объекта JSON в данном случае будет таким: {"and": [ARRAY]}, где ARRAY (массив) состоит из двух конструкций формата { "операция сравнения": { VARIABLE } }. Чтобы логическое выражение правильно считалось функцией, обе переменные должны объединяться с помощью конкатенации, поэтому нужно записать каждую переменную (VARIABLE) со знаками +:

   >={data:+stringDate1UTC+} и <{data:+stringDate2UTC+}.

   В итоге, второй параметр функции выглядит так:

   {\"AND\":[{\">=\":{\"data\":"+stringDate1UTC+"}},{\"<\":{\"data\":"+stringDate2UTC+"}}]}

Вся конструкция фильтра выглядит так:

```
//язык OM
date1:uint8 = DateTime.Parse("09.02.2024");
ddate1UTC: uint8 = DateTime.ToUtc(date1);
stringDate1UTC:string = String.ToString(date1UTC);
date2:uint8 = DateTime.Parse("10.02.2024");
```

```
date2UTC: uint8 = DateTime.ToUtc(date2);
stringDate2UTC:string = String.ToString(date2UTC);
Alarms_1.SetAdvancedFilterAsJson("active_time", "{\"AND\":[{\">=\":
{\"data\":"+stringDate1UTC+"}},{\"<\":{\"data\":"+stringDate2UTC+"}}]}");
```

Фильтрация по нескольким столбцам одновременно

ПРИМЕР

Показать все сообщения о событиях с важностью **«750»**, содержащие слово **«уровень»**.

Отдельными строками запишите условия для столбца **Важность** (идентификатор severity) и столбца Сообщение (идентификатор message). В каждом случае условие фильтрации (второй параметр функции) составьте по формату { "операция сравнения": { VARIABLE } }. В первом случае VARIABLE (переменная) - data: уровень, во втором - data: 750. Операцию сравнения в данных случаях можно не ставить.

В итоге, вся конструкция фильтра выглядит так:

```
//язык ОМ
Alarms_1.SetAdvancedFilterAsJson("message", "{\"data\":\"*уровень*\"}");
Alarms_1.SetAdvancedFilterAsJson("severity","{\"data\":750}");
```

CancelHistoryQuery

```
void CancelHistoryQuery()
```

Прерывает запрос истории без подтверждения, если он в процессе выполнения. Если запрос не в процессе выполнения, функция не сработает.

ReloadHistory

```
void ReloadHistory()
```

Перезапрашивает историю за заданный ранее интервал времени, который был установлен на панели инструментов исторического режима или с помощью функции SetHistoricalInterval (стр. 63).

SetAcknowledgeAvailability

```
void SetAcknowledgeAvailability(string Columnid string Values)
```

Позволяет указать зоны, сообщения о событиях которых можно квитировать и комментировать. Для сообщений остальных зон квитирование и комментирование блокируются.

Параметры

Columnid	string	Идентификатор столбца.
Values	string	Список зон в формате: [\"Area1\",\"Area2\"]

Примеры

ПРИМЕР

Блокировать квитирование и комментирование для всех зон, кроме зоны **Discrete**.

```
Alarms_1.SetAcknowledgeAvailability("object_name","[\"Discrete\"]");
```

SetAdvancedRequestFilterAsJson

```
bool SetAdvancedRequestFilterAsJson(string StringJSON)
```

Применяет фильтр запроса (в формате JSON) исторических данных.

Параметры

StringJSON	string	Условия фильтрации. Задаются в виде строки - объекта JSON. Правила формирования условий фильтрации описаны ниже.

Возвращаемое значение

«true»	Фильтр запроса успешно установлен.
«false»	Неудачный разбор выражения фильтрации, фильтр не установлен.

Правила формирования условий фильтрации

Условие фильтрации для столбца нужно записывать в виде объекта JSON с использованием операторов сравнения и/или логических операторов. Такие выражения могут содержать всего одно условие фильтрации (одиночный объект JSON: {"ключ": "значение"}), либо быть сложными и включать сразу несколько условий (составной объект JSON, состоящий из нескольких объектов).

Когда вы составляете логическое выражение, помните, что внутри каждого столбца содержатся данные определенного типа (стр. 60) (в столбце **Сообщение** - строковые данные, в столбце **Важность** - число и т.д.), поэтому подбирайте операторы, работающие с этим типом. Ниже для всех операторов сказано с какими столбцами они работают.

Операторы сравнения

Операторы сравнения нужны для сравнения значений в фильтруемом столбце с какими-либо произвольными значениями. Такие операторы вы можете использовать для фильтрации по столбцам, содержащим число (стр. 60) или время (стр. 60).

<	меньше
>	больше
<=	меньше или равно
>=	больше или равно
==	равно
!=	не равно

Логические операторы

Логические операторы нужны для сложных выражений, когда несколько условий объединяются в одно условие, а также для создания отрицательного условия фильтрации.

Символьное написание	Англоязычное написание	Описание	Для столбцов
&	AND, and	логическое "и"	Можно использовать для фильтрации по любому столбцу.
|	OR, or	логическое "или"	Можно использовать для фильтрации по любому столбцу.
!	NOT, not	логическое "не"	Можно использовать только для столбцов, содержащих строку.

Логические выражения в формате JSON

Форматы составного объекта JSON

В зависимости от используемой операции составной объект JSON может иметь формат:

- {"and": [ARRAY]}
- {"or": [ARRAY]}
- { "not": { OBJECT } }
- { "операция сравнения": { VARIABLE } }

VARIABLE	Переменная. Имеет структуру: <имя переменной>: <значение переменной>. В имени переменной вы можете написать **«text»** и тогда в качестве значения такой переменной укажите значение, которое вы видите в таблице. Либо можете написать **«data»** и тогда в имени переменной укажите внутреннее значение, хранящееся в источнике данных. Например, для столбца **Квитировано** (ack) переменная **«text»** имеет значение **«Да»**, а переменная **«data»** - значение **«true»**.
OBJECT	Переменная или составной объект JSON любого из указанных выше форматов.
ARRAY	Массив. Может состоять из переменных и составных объектов JSON (любого формата) в различных комбинациях.

Правила составления логических выражений в формате JSON

- Операцию сравнения == можно не указывать. В таком случае объект будет иметь формат { VARIABLE }.
- В столбцах с датой и временем значение нужно указывать в формате UTC. Для начала переведите ваше строковое значение даты в метку времени формата uint8, а затем полученное значение переведите в формат UTC. Для перевода вы можете использовать функции Parse и ToUtc языка Alpha.Om.
- Для столбцов с текстом:
  - Условия фильтрации для столбцов с текстом можно записывать без учета регистра.
  - Чтобы записать условие фильтрации, при котором столбец с текстом фильтруется по нескольким находящимся не рядом словам, используйте оператор and (&).

- Условие фильтрации записывается в виде строки в кавычках, поэтому для правильной работы фильтра в языке OM экранируйте все остальные кавычки в строке символом \. В языке JavaScript этого делать не нужно.

Примеры

ПРИМЕР

Запросить события с:

- важностью (столбец **severity**) больше **«10»**,
- сообщением (столбец **message**), содержащим текст **«поставлен»** или с именем источника (столбец **source**), не соответствующим **«TU_212»**. Записать результат выполнения функции в текстовое поле.

```
Filter: string = "{ "AND": [ { ">": { "severity": 10 } }, { "OR": [ { "message":
"*поставлен*" }, { "NOT": { "source": "TU_212" } } ] } ] }";
Result: bool = Alarms_1.SetAdvancedRequestFilterAsJson(Filter);
TextEdit_1.Text = Str.ToString(Result);
```

SetHistoricalInterval

```
bool SetHistoricalInterval(uint8 t1, uint8 t2)
```

Устанавливает интервал отображения событий в историческом режиме и запрашивает данные за этот интервал.

Параметры

t1	uint8	Начало временного интервала.
t2	uint8	Окончание временного интервала.

Возвращаемое значение

«true»	Интервал установлен и запрос прошел успешно.
«false»	Интервал задан в неправильном формате.

Примеры

ПРИМЕР

Запросить данные за время с **«20.09.2022 10:10:20»** по **«21.09.2022 10:10:20»** и вывести сообщение о корректности ввода интервала.

```
fromDate: uint8 = DateTime.Parse ("20.09.2022 10:10:20");
toDate: uint8 = DateTime.Parse ("21.09.2022 10:10:20");
Result: bool = Alarms_1.SetHistoricalInterval(fromDate, toDate);
  if (Result){
    TextEdit.Text = "Исторический интервал успешно установлен";
     }
  else {
     TextEdit.Text = "Задан неверный исторический интервал";
       }
```

SetSize

```
void SetSize(float8 coordX , float8 coordY , float8 width , float8 height)
```

Задает размер объекта типа **Alarms** и позиционирует объект на форме.

Параметры

coordX	float8	Количество пикселей от левого края формы до левого края объекта.
coordY	float8	Количество пикселей от верхнего края формы до верхнего края объекта.
width	float8	Ширина объекта (в пикселях).
height	float8	Высота объекта (в пикселях).

Примеры

ПРИМЕР

Расположить таблицу событий с шириной «**1000**» и высотой «**500**» в левом верхнем углу формы.

```
Alarms_1.SetSize(0, 0, 1000, 500);
```

SkipCurrentSound

```
void SkipCurrentSound()
```

Останавливает воспроизведение текущего звука.

RequestRecentEvents

```
void RequestRecentEvents(float8 intervalDuration, bool onlyNotAcked, string
requestAdvancedFilterAsJson)
```

Подгружает исторические события, возникшие за указанный интервал времени, в таблицу оперативного режима. Подгруженные сообщения можно также, как и оперативные, настраивать (цвет, мигание и т.д.) и квитировать.

ПРИМЕЧАНИЕ

Звуки подгружаемых событий не проигрываются.

ОБРАТИТЕ ВНИМАНИЕ

Подгружая исторические события в таблицу оперативного режима, следует помнить, что для таблицы оперативного режима установлено максимальное количество отображаемых обычных событий (свойство MaxVisibleOrdinaryEvents (стр. 24)).

Параметры

intervalDuration	float8	Интервал времени (в часах), за который будут подгружены исторические события. Можно указывать нецелое значение. Например, **«0,5»**
onlyNotAcked	bool	Вид подгружаемых событий: › **«true»** - подгружаются только неквитированные события › **«false»** - подгружаются любые события
requestAdvancedFilterAsJson	string	Условия фильтрации подгружаемых событий. Задаются в виде строки - объекта JSON. Фильтр работает по принципу функции SetAdvancedRequestFilterAsJson (стр. 60)

Примеры

ПРИМЕР

Подгрузка в таблицу оперативного режима всех неквитированных событий с важностью 600, возникшие за последние сутки.

```
Filter:string = "{\"==\":{\"severity\":600}}";
  Alarms_1.RequestRecentEvents(24,true,Filter);
```

2. Alarms_Table

Тип, который представляет собой таблицу сообщений о событиях (оперативные и исторические) без панели инструментов, статусной строки и возможности проигрывания звуков событий.

Для работы с типом просто добавьте его на форму и укажите источник данных в свойстве Ссылка на источник AP (стр. 66). После этого в таблице начнут отображаться сообщения о событиях.

Далее вы можете связать этот тип с другими типами для более тонких настроек.

2.1. Свойства

2.1.1. Ссылка на источник AP (init_ApSource)

Ссылка на экземпляр типа **Источник AP**. Используется для указания источника данных для экземпляра типа **Alarms_Table**.

2.1.2. Фильтр подписки (SubscriptionAdvancedFilter)

Работает также, как свойство SubscriptionAdvancedFilter (стр. 22) типа **Alarms**.

2.1.3. Активный режим работы (0-оперативный, 1 - исторический) (ActiveMode)

Работает также, как свойство ActiveMode (стр. 24) типа **Alarms**.

2.1.4. OnPrint

Ссылка на команду, которая вызывается при печати документа.

Выводит в журнал сообщение об успешной печати (если в команде заложены настройки вывода сообщения).

2.1.5. OnSave

Ссылка на обработчик события, которое срабатывает при успешном сохранении архива событий (`*.avz`).

Выводит полный путь до сохраненного архива, либо текст ошибки, если архив не сохранился (если в обработчике заложены настройки вывода сообщения).

2.1.6. OnLoad

Ссылка на обработчик события, которое срабатывает при загрузке архива событий (`*.avz`).

Выводит полный путь до сохраненного архива, либо текст ошибки, если загрузка не удалась (если в обработчике заложены настройки вывода сообщения).

2.1.7. OnAck

Ссылка на обработчик события, которое срабатывает при включении внешнего квитирования.

2.1.8. OnMouseRightRelease

Ссылка на обработчик события, которое срабатывает при нажатии правой кнопкой мыши в рабочей области таблицы. Используется для построения контекстного меню.

2.1.9. OnHistoryLoad

Ссылка на команду, которая вызывается при любом историческом запросе.

Выводит сообщение о завершении загрузки истории (если в команде заложены настройки вывода сообщения).

2.1.10. OnOperativeLoad

Ссылка на команду, которая вызывается при любой подгрузке оперативных сообщений (Refresh, вызов функции RequestRecentEvents, загрузка оперативной таблицы при первом запуске или переходе из исторического режима в оперативный).

Выводит сообщение о завершении подгрузки (если в команде заложены настройки вывода сообщения).

2.1.11. OnError

Ссылка на обработчик события, которое срабатывает при возникновении любых исключительных ситуаций.

Выводит текст сообщения об ошибке (если в обработчике заложены настройки вывода сообщения).

2.1.12. OnActionInSourceColumn

Ссылка на обработчик события, которое срабатывает при клике на столбце **Источник**.

Выводит ObjectId источника выделенного сообщения (если в обработчике заложены настройки вывода сообщения).

2.1.13. OnExport

Ссылка на обработчик события, которое срабатывает при успешном сохранении файла с данными (`*.pdf`, `*.xls` или `*.csv`).

Выводит полный путь до экспортированного файла, либо текст ошибки, если файл не сохранился (если в обработчике заложены настройки вывода сообщения).

2.1.14. Параметры вызова экранной клавиатуры (pScreenKeyboard)

Ссылка на экземпляр типа **Параметры вызова экранной клавиатуры** (из состава внешнего модуля **CommonLib**), хранящий настройки вызова экранной клавиатуры.

2.1.15. Разрешения пользователя (pPermissions)

Ссылка на экземпляр типа **Разрешения пользователя** (из состава внешнего модуля **Alarms**), хранящий настройки доступа пользователей к функциям Alpha.HMI.Alarms.

2.1.16. Тема оформления (init_Theme)

Ссылка на экземпляр типа **Тема офрмления** (из состава внешнего модуля **CommonLib**), хранящий настройки визуального оформления Alpha.HMI.Alarms.

2.1.17. Шаблоны (pTemplates)

Ссылка на экземпляр типа **Текстовые шаблоны** (из состава внешнего модуля **CommonLib**), хранящий шаблоны, которые будут использоваться для подстановки в колонтитулы.

2.1.18. Статистика (pEventStat)

Ссылка на экземпляр типа **EventStatistics** (из состава внешнего модуля **Alarms**), хранящий детальную статистику по событиям.

2.1.19. OnSettingsChanged

Ссылка на команду, которая вызывается при изменении пользовательских настроек (например, на вкладках окна **Параметры**, а также некоторых других настроек, доступных из интерфейса приложения Alpha.HMI.Alarms). Данные об этих настройках сохраняются в типы из семейства **Config**, а также в конфигурационные файлы формата `*.json`.

2.1.20. pConfigTable

Ссылка на экземпляр типа **ConfigTable** из семейства **Config**, хранящий настройки вкладки **Вид таблицы** окна **Параметры**.

2.1.21. pConfigAcknowledge

Ссылка на экземпляр типа **ConfigAcknowledge** из семейства **Config**, хранящий настройки вкладки **Квитирование** окна **Параметры**.

2.1.22. pConfigPresetFilters

Ссылка на экземпляр типа **ConfigPresetFilters** из семейства **Config**, хранящий настройки вкладки **Фильтры** окна **Параметры**.

2.1.23. pConfigEvents

Ссылка на экземпляр типа **ConfigEvents** из семейства **Config**, хранящий настройки вкладки **Настройка событий** окна **Параметры**.

2.1.24. pConfigExport

Ссылка на экземпляр типа **ConfigExport** из семейства **Config**, хранящий настройки вкладки **Экспорт** окна **Параметры**.

2.1.25. pConfigSort

Ссылка на экземпляр типа **ConfigSort** из семейства **Config**, хранящий настройки сортировки событий.

2.1.26. pConfigSession

Ссылка на экземпляр типа **ConfigSession** из семейства **Config**, хранящий имя последнего использованного принтера.

2.1.27. pConfigSupression

Ссылка на экземпляр типа **ConfigSupression** из семейства **Config**, хранящий настройки вкладки **Подавления и блокировки** окна **Параметры**.

2.1.28. pConfigViewFilter

Ссылка на экземпляр типа **ConfigViewFilter** из семейства **Config**, хранящий настройки вкладки **Фильтр отображения** окна **Параметры**.

2.1.29. pConfigViewFilter

Ссылка на экземпляр типа **ConfigViewFilter** из семейства **Config**, хранящий настройки вкладки **Фильтр отображения** окна **Параметры**.

2.1.30. pConfigPrint

Ссылка на экземпляр типа **ConfigPrint** из семейства **Config**, хранящий настройки вкладки **Печать** окна **Параметры**.

2.1.31. pConfigExportView

Ссылка на экземпляр типа **ConfigExportView** из семейства **Config**, хранящий настройки окна предпросмотра экспорта.

2.1.32. pConfigToolbar

Ссылка на экземпляр типа **ConfigToolbar** из семейства **Config**, хранящий настройки вкладки **Панель инструментов** окна **Параметры**.

2.1.33. pConfigMenu

Ссылка на экземпляр типа **ConfigMenu** из семейства **Config**, хранящий настройки вкладки **Контекстное менюы** окна **Параметры**.

2.1.34. pConfigCommentPattern

Ссылка на экземпляр типа **ConfigCommentPattern** из семейства **Config**, хранящий настройки вкладки **Комментарии** окна **Параметры**.

2.1.35. pConfigSounds

Ссылка на экземпляр типа **ConfigSounds** из семейства **Config**, хранящий настройки вкладки **Звуки событий** окна **Параметры**.

2.1.36. pConfigPrintView

Ссылка на экземпляр типа **ConfigPrintView** из семейства **Config**, хранящий настройки окна предпросмотра печати.

2.2. Функции

AckSelected

Работает также, как функция AckSelected (стр. 27) типа **Alarms**.

AckAll

Работает также, как функция AckAll (стр. 27) типа **Alarms**.

AckExt

```
int4 AckExt(bool showDlg)
```

Выполняет внешнее квитирование.

Параметры

showDlg	bool	Вызов диалогового окна при внешнем квитировании: › **«true»** - включить вызов диалога при внешнем квитировании; › **«false»** - отключить вызов диалога при внешнем квитировании.

Возвращаемое значение

«-2»	Пользователь не зарегистрирован или нет разрешения на квитирование по свойству **Доступность функций квитирования** или праву **Acknowledgment** в системе безопасности.
«-1»	Текущий режим работы - исторический.
«-3»	При успешном квитировании.
«0»	В списке событий отсутствуют неквитированные события.

Примеры

ПРИМЕР

Выполнить внешнее квитирование с вызовом диалога.

```
Alarms_Table_1.AckExt(true);
```

2.2.1. AddComment

```
void AddComment()
```

Открывает окно ввода комментария для выделенного события.

2.2.2. ClearOperativeEventList

Работает также, как функция **ClearOperativeEventList** (стр. 29) типа **Alarms**.

2.2.3. ClearAcknowledgedEvents

Работает также, как функция **ClearAcknowledgedEvents** (стр. 27) типа **Alarms**.

SetAdvancedRequestFilterAsJson

Работает также, как функция **SetAdvancedRequestFilterAsJson** (стр. 60) типа **Alarms**.

SetHistoricalInterval

Работает также, как функция **SetHistoricalInterval** (стр. 63) типа **Alarms**.

RequestRecentEvents

Работает также, как функция **RequestRecentEvents** (стр. 65) типа **Alarms**.

CancelHistoryQuery

Работает также, как функция **CancelHistoryQuery** (стр. 58) типа **Alarms**.

ReloadHistory

Работает также, как функция **ReloadHistory** (стр. 58) типа **Alarms**.

SaveToFile

```
void SaveToFile(string filePath)
```

Сохраняет файл данных с указанным именем по указанному пути.

Параметры

filePath	string	путь к папке сохранения

Примеры

ПРИМЕР

Сохранить файл данных с именем **«Alarms_20-03-25»** по пути `D:\Projects Alarms`.

```
Alarms_Table_1.SaveToFile("D:\\Projects Alarms\\Alarms_20-03-25.avz");
```

SaveToFolder

```
void SaveToFolder(string folderPath)
```

Сохраняет два файла данных (`alarms.inf` и `alarms.json`) по указанному пути.

Параметры

folderPath	string	путь к папке сохранения

Примеры

ПРИМЕР

Сохранить файлы данных по пути `D:\\Projects Alarms`.

```
Alarms_Table_1.SaveToFolder("D:\\Projects Alarms");
```

LoadFromFile

```
void LoadFromFile(string filePath)
```

Подгружает данные из указанного файла.

Параметры

filePath	string	путь к папке с файлом данных

Примеры

ПРИМЕР

Подгрузить файл данных с именем **«`Alarms_20-03-25`»**, хранящийся по пути `D:\\Projects Alarms`.

```
Alarms_Table_1.LoadFromFile("D:\\Projects Alarms\\Alarms_20-03-25.avz");
```

LoadFromFolder

```
void LoadFromFolder(string folderPath)
```

Подгружает файл данных из указанной папки.

Параметры

folderPath	string	путь к папке с файлом данных

Примеры

ПРИМЕР

Подгрузить файл данных, хранящийся по пути `D:\\Projects Alarms`.

```
Alarms_Table_1.LoadFromFolder("D:\\Projects Alarms");
```

2.2.4. GetSelectedSource

```
string GetSelectedSource()
```

Возвращает тег источника сообщения, выделенного в таблице.

SetSnapshotState

Работает также, как функция **SetSnapshotState** (стр. 48) типа **`Alarms`**.

LoadConfiguration

Работает также, как функция **LoadConfiguration** (стр. 29) типа **`Alarms`**.

SaveConfiguration

```
void SaveConfiguration(string folder)
```

Сохраняет файлы конфигурации в указанную папку. Если папки не существует, она будет создана автоматически.

Параметры

folder	string	путь к папке сохранения

Примеры

ПРИМЕР

Сохранить конфигурацию по пути `D:\\Alarms_config`.

```
Alarms_Table_1.SaveConfiguration("D:\\Alarms_config");
```

SetAcknowledgeAvailability

Работает также, как функция **SetAcknowledgeAvailability** (стр. 58) типа **`Alarms`**.

3. Alarms_Sounds

Тип, который не отображается на форме и предназначен только для воспроизведения звуков событий.

3.1. Свойства

3.1.1. ApSource

Источник данных для экземпляра типа **`Alarms_Sounds`**.

3.1.2. ClearSoundQueryTag

Работает также, как свойство **ClearSoundQueryTag** (стр. 19) типа **`Alarms`**.

3.1.3. SubscriptionAdvancedFilter

Работает также, как свойство **SubscriptionAdvancedFilter** (стр. 22) типа **`Alarms`**.

3.1.4. Volume

```
int4 Volume
```

Уровень громкости проигрываемых звуков.

3.1.5. OnPlayingSounds

Ссылка на команду, которая вызывается при запуске воспроизведения звуков.

Выводит сообщение о запуске воспроизведения (если в команде заложены настройки вывода сообщения).

3.1.6. OnFinishedPlayingSounds

Ссылка на команду, которая вызывается при окончании воспроизведения звуков.

Воспроизведение считается завершенным когда:

- Проиграли все звуки в очереди.
- Нажата кнопка **`Очистить очередь звуков`**.
- Квитированы все сообщения, от которых играли звуки.

Выводит сообщение об окончании воспроизведения (если в команде заложены настройки вывода сообщения).

3.1.7. OnSoundPlayed

Ссылка на обработчик события, срабатывающего при проигрывании конкретного звука.

Выводит в журнал (если в обработчике заложены настройки вывода сообщения):

- путь к звуковому файлу относительно папки `resources`;
- сообщение об ошибке воспроизведения (если файл испорчен);
- сообщение об отсутствии файла, если файл не найден.

3.1.8. Настройки звуков (pSounds)

Ссылка на экземпляр типа **`ConfigSounds`** из семейства **`Config`**, хранящий пользовательские звуковые настройки (группы важности, приоритеты, количество проигрывания и т. д.).

3.1.9. Настройки подключения (pConnection)

Ссылка на экземпляр типа **`ConfigConnection`** из семейства **`Config`**, хранящий параметры подключения из вкладки **Подключение** окна **Параметры**.

3.1.10. Настройки вида таблицы (pConfigTable)

Ссылка на экземпляр типа **`ConfigTable`** из семейства **`Config`**, хранящий настройки вкладки **Вид таблицы** окна **Параметры**.

3.2. Функции

ClearSoundQueue

Работает также, как функция **ClearSoundQueue** (стр. 29) типа **`Alarms`**.

ReloadSoundQueue

```
void ReloadSoundQueue()
```

Перезапускает звуковую очередь.

SkipCurrentSound

```
void SkipCurrentSound()
```

Останавливает воспроизведение текущего звука.

4. EventStatistics

Тип, который не отображается на форме и предназначен для вывода детальной статистики по событиям, которая в приложении выводится в статусной строке.

4.1. Свойства

Включен подсчёт количества по важности (Enabled)

```
bool Enabled
```

Уведомляющее поле. Позволяет вывести состояние функции подсчета статистических данных.

Значение

«true»	Включен подсчет статистических данных.
«false»	Отключен подсчет статистических данных.

Количество отображаемых (CountView)

```
int4 CountView
```

Уведомляющее поле. Позволяет вывести количество всех отображаемых в таблице сообщений о событиях.

Количество загруженных (CountTotal)

```
int4 CountTotal
```

Уведомляющее поле. Позволяет вывести количество всех загруженных с сервера сообщений о событиях.

Количество по важности (CountViewBySeverity)

```
int4 CountViewBySeverity
```

Уведомляющее поле. Позволяет вывести количество сообщений с одинаковым значением важности в формате: **«[Приоритет: Количество]»**.

Имя открытого файла (FileName)

```
string FileName
```

Уведомляющее поле. Позволяет вывести имя файла, из которого загружены события.

Текст ошибки (ErrorString)

```
string ErrorString
```

Уведомляющее поле. Позволяет вывести текст ошибки при неудачной загрузке файла с событиями.

5. Permissions

Тип, который не отображается на форме и предназначен для ограничения прав пользователей.

5.1. Свойства

5.1.1. Имя приложения с правами безопасности (SecurityApplication)

Работает также, как свойство SecurityApplication (стр. 15) типа `Alarms`.

5.1.2. Использовать сервер безопасности для определения прав (UseSecurity)

Работает также, как свойство UseSecurity (стр. 19) типа `Alarms`.

5.1.3. Доступность редактирования настроек (EditingSettingsAvailable)

Работает также, как свойство EditingSettingsAvailable (стр. 11) типа `Alarms`.

5.1.4. Доступность сохранения данных в табличный файл (ExportDataAvailability)

Работает также, как свойствоExportDataAvailability (стр. 11) типа `Alarms`.

5.1.5. Доступность фильтрации (FilterAvailable)

Работает также, как свойство FilterAvailable (стр. 11) типа `Alarms`.

5.1.6. Доступность просмотра оперативных событий (CurrentEventsAvailable)

Работает также, как свойство CurrentEventsAvailable (стр. 12) типа `Alarms`.

5.1.7. Доступность просмотра истории событий (EventHistoryAvailable)

Работает также, как свойство EventHistoryAvailable (стр. 13) типа `Alarms`.

5.1.8. Доступность управления печатью (PrintingAvailable)

Работает также, как свойство PrintingAvailable (стр. 13) типа `Alarms`.

5.1.9. Видимость команд сортировки (EventTableSortAvailability)

Работает также, как свойство EventTableSortAvailability (стр. 13) типа `Alarms`.

5.1.10. Доступность файловой системы (FileSystemAccess)

Работает также, как свойство FileSystemAccess (стр. 18) типа `Alarms`.

5.1.11. Доступность функций квитирования (AcknowledgmentAvailable)

Работает также, как свойство AcknowledgmentAvailable (стр. 9) типа `Alarms`.

5.1.12. Доступность функций квитирования (AcknowledgmentAvailable)

Работает также, как свойство AcknowledgmentAvailable (стр. 9) типа `Alarms`.

5.1.13. Доступность кнопок управления проигрыванием звуков (SoundPlaybackManagmentAvailable)

Работает также, как свойство SoundPlaybackManagmentAvailable (стр. 12) типа `Alarms`.

5.1.14. Доступность кнопок функции Снимок (SnapshotAvailable)

Работает также, как свойствоSnapshotAvailable (стр. 12) типа `Alarms`.

5.1.15. Разрешение использования блокировки и подавления (SuppressionAvailable)

Работает также, как свойство SuppressAndBlock (стр. 25) типа `Alarms`.

Доступность выбора принтера (PrinterSelectingAvailable)

```
bool PrinterSelectingAvailable
```

Разрешает/запрещает открытие окна со списком доступных принтеров перед печатью.

Значение

«false»	Запретить открытие окна со списком доступных принтеров.
«true»	Разрешить открытие окна со списком доступных принтеров.

Перечень допускаемых принтеров (AllowedPrinters)

```
string AllowedPrinters
```

Список принтеров, разрешенных к использованию.

5.1.16. Разрешить действия в столбце источник события (AllowActionInSourceColumn)

Работает также, как свойство **AllowActionInSourceColumn** (стр. 14) типа **`Alarms`**.

5.1.17. Возможность изменения вида пользовательского интерфейса (GUIModificationAvailable)

Работает также, как свойство **GUIModificationAvailable** (стр. 23) типа **`Alarms`**.

5.2. Функции

LogOn

Работает также, как функция **LogOn** (стр. 31) типа **`Alarms`**.

LogOff

Работает также, как функция **LogOff** (стр. 31) типа **`Alarms`**.

Программный комплекс Альфа платформа

Alpha.HMI.CommonLib 2.4

Руководство пользователя

Редакция	Соответствует версии ПО
1	2.4.0

© АО «Атомик Софт», 2015-2025. Все права защищены.

Авторские права на данный документ принадлежат АО «Атомик Софт». Копирование, перепечатка и публикация любой части или всего документа не допускается без письменного разрешения правообладателя.